令和 7 年度 修士論文

Issuer-Hiding Verifiable Credentials の実現の検討

Consideration of Issuer-Hiding Verifiable Credentials

水野　重弦

早稲田大学　基幹理工学研究科

情報理工・情報通信専攻

5124F101-4

提出日　　：令和 8 年 1 月 26 日

研究指導名：暗号プロトコル研究

指導教員　：佐古 和恵 教授

概要

World Wide Web Consortium (W3C) が標準化を進める Verifiable Credentials (VC) は，ユーザが必要な属性のみを選択的に開示する「選択的開示」が可能な属性情報の証明書である．ただし，証明書の検証のために VC の発行元の公開鍵を参照する際，その鍵に紐づく発行元自体も特定される．発行元の情報がユーザの属性情報となる場合，検証者に属性を推測される可能性がある．例えば，早稲田大学の学生に対して学生証が VC として発行されていた場合，発行元が「早稲田大学」であるということがわかると，学生が早稲田大学の学生であるということがわかってしまう．そこで，VC でも用いることが一部検討されている Anonymous Credentials（AC）において，発行元の公開鍵を秘匿したまま証明書の検証を可能にする Issuer-Hiding AC が提案されている．Issuer-Hiding AC では，証明書の検証を行う検証者が受け入れ可能な発行元の公開鍵をポリシーに記載（ポリシー作成）し，ユーザはこのポリシー内に実際の証明書の発行元が含まれていることを証明する（証明生成）ことで，検証者が発行元を匿名にされたまま証明書の検証（証明検証）をすることを可能とする．しかし，既存の Issuer-Hiding AC では，標準化されていない群構造維持署名方式を前提としていたり，計算コストやデータサイズが大きいなどの問題がある．また，計算能力に制限をかけた計算モデルである Generic Group Model を仮定においており，計算能力に制限をかけない標準モデルのもとで安全性が証明することができれば，なお良い．そこで本稿では標準化が進んでいる BBS 署名を利用し，公開鍵の認証局への登録時に対応する秘密鍵を所持していることの証明も示す Registered Key Model を仮定することで標準モデルの下でも Issuer-Hiding AC を構成できることを示した．また，既存の Issuer-Hiding AC の 3 方式と提案した Issuer-Hiding AC の方式を計算モデル，計算コストやデータサイズの面で比較・評価した．その結果，提案方式は既存方式と比較してポリシー作成時の計算コストが最も低く，証明生成や証明検証の処理速度も高速であることを確認した．また，提案方式は比較した 4 方式の中で唯一，計算モデルが標準モデルであることから，セキュリティ面でも優れていると考えられる．さらに，本稿では Issuer-Hiding AC を用いた Verifiable Credentials を構成するために，VC と Issuer-Hiding VP のデータフォーマットを提案した．

目次

1 はじめに 6

2 準備 8

2.1 双線形写像 8

2.2 計算モデル 8

2.2.1 Generic Group Model (GGM) 8

2.2.2 Standard Model（標準モデル） 8

2.2.3 Algebraic Group Model (AGM) 9

2.3 デジタル署名 9

2.4 コミットメント付きデジタル署名 10

2.5 BBS 署名 11

2.6 PS 署名 12

2.7 Groth 署名 13

2.8 ゼロ知識証明 (Zero-Knowledge Proofs) 13

2.9 シグマプロトコル 14

2.10 非対話ゼロ知識証明 (NIZK) 15

2.11 知識証明（Proofs of Knowledge）と抽出可能性 (Extractability) 15

2.12 Simulation-Sound Extractable NIZK 16

2.13 匿名認証 (Anonymous Credentials) 16

2.14 Issuer-Hiding AC の概要 17

2.14.1 シンタックス 17

2.14.2 安全性 17

2.15 登録鍵モデル (Registered Key Model) 18

3 Issuer-Hiding AC 19

3.1 Issuer-Hiding AC の一般的構成 19

3.1.1 Issuer-Hiding AC の一般的な構成 19

3.1.2 群構造維持署名を用いない Issuer-Hiding AC の構成 21

3.1.3 安全性証明 21

3.2 Bobolz らによる Issuer-Hiding AC 23

3.2.1 Bobolz らの Issuer-Hiding AC の改良版 24

3.3 Sanders らによる Issuer-Hiding AC 25

3.4 Katz らによる Issuer-Hiding AC 27

4 BBS 署名を用いた Issuer-Hiding AC の提案 29
4.1 Issuer-Hiding AC の構成 29
4.2 BBS 署名を用いた Issuer-Hiding AC の構成の安全性の証明 31

5 比較・評価 33
5.1 実行環境と設定 33
5.2 評価結果 33
5.2.1 安全性証明モデル 33
5.2.2 理論的な計算コストとデータサイズ 33
5.2.3 ark_bls12_381 のベンチマーク 37
5.2.4 実行時間 38
5.3 考察 41
5.3.1 公開パラメータ生成と鍵生成 41
5.3.2 証明書の作成と検証 43
5.3.3 ポリシーの作成と検証 43
5.3.4 証明の生成と検証 44
5.3.5 データサイズ 45
5.4 まとめ 45

6 Verifiable Credentials 46
6.1 Verifiable Credentials の概要 46
6.1.1 データモデル 46
6.1.2 VC の登場人物 46
6.2 選択的開示 47
6.2.1 連結不可能な選択的開示 47
6.2.2 連結可能な選択的開示 47
6.3 Issuer-Hiding Verifiable Credentials の検討 48
6.3.1 Issuer-Hiding Verifiable Credentials のフォーマットの提案 48

7 まとめと今後の課題 51

謝辞 52

図目次

1 EUF-CMA 実験 10
2 SUF 実験 12
3 Issuer-Hiding AC の一般的構成 19
4 Unforgeability の実験 20
5 Unlinkability の実験 21
6 証明書生成時間の比較 39
7 証明書検証時間の比較 40
8 ポリシー作成時間の比較 40
9 ポリシー検証時間の比較 41
10 属性数を変えた時の証明生成時間の比較 41
11 検証者が受け入れ可能な発行元数を変えた時の証明生成時間の比較 41
12 属性数を変えた時の証明検証時間の比較 42
13 検証者が受け入れ可能な発行元数を変えた時の証明検証時間の比較 42
14 凡例 42
15 Verifiable Credentials のシステム全体の概要図 46

表目次

1 公開パラメータ生成と発行元の鍵生成の計算コストの比較 34
2 証明書生成と検証の計算コストの比較 34
3 ポリシー作成の計算コストの比較 35
4 ポリシー検証の計算コストの比較 35
5 証明生成の計算コストの比較 35
6 証明検証の計算コストの比較 36
7 各方式の公開パラメータ，発行元の鍵，証明書のデータサイズ比較 36
8 各方式のポリシーと提示トークンのデータサイズ比較 37
9 ark_bls12_381 での各処理 1000 回分の実行時間 37
10 公開パラメータの生成と発行元の鍵生成の実行時間の比較 38

1　はじめに

近年，デジタル社会の発展に伴い，個人情報の保護がますます重要となっている．その中で，Verifiable Credentials（VC）への関心が高まっている．VC は，World Wide Web Consortium (W3C) によって標準化が進められているデジタル証明書のフレームワークであり [1]，選択的開示をサポートしている．Anonymous Credentials (AC) は，ユーザが自身の属性情報の一部のみを選択的に開示できる技術の一つであり，その技術の一部を標準化したり，データモデル等を整えたものが VC である．そのため，AC の研究は VC の今後の発展においても非常に重要である．

選択的開示は，ユーザが必要な属性情報のみを検証者に開示することで，プライバシーを保護する技術である．この際，ユーザは開示した属性情報が発行元から発行された証明書に基づいていることを証明し（証明生成），検証者はその証明を検証することで開示された属性を受け入れるか決める（証明検証）．たとえば，酒屋で年齢確認のために不必要な住所などの情報を秘匿し，年齢情報のみを開示したとしても，検証者は受け入れることができる．AC の研究は盛んであり，様々な署名方式を使った AC が提案されている．中でも BBS 署名 [2] を用いた AC は，効率的な証明生成・検証が可能であり，署名方式が Internet Research Task Force (IRTF) によって標準化が進められている [3] ほか，W3C で標準化が進められている VC の標準仕様 [1] においても採用されている．

ただし，VC や一般的な AC では発行元の公開鍵を用いて証明検証を行うため，検証者が発行元の公開鍵を参照する際に，その鍵に紐づく発行元自体も特定される．発行元の情報からユーザの属性情報が推測される場合に，プライバシーを十分に保護できない問題が生じる．たとえば，大学が発行する学生証を VC あるいは AC として利用する場合を考える．学割を利用する際に本来必要な情報は「大学生である」あるいは「学生である」という情報のみである．しかし，発行元の公開鍵を用いて証明検証を行うため，発行元を明かす必要がある．この際，検証者は「発行元の公開鍵」が認められる発行元の公開鍵か確認するため，早稲田大学が発行している学生証を提示する場合は，発行元が早稲田大学であるということを知る必要がある．すると「早稲田大学が発行している学生証だから，この人は早稲田大学の学生なのだな」と検証者に本来必要ない情報を知られてしまう．この問題を解決する方法として，発行元の公開鍵を秘匿したままユーザが証明を生成し，検証者が検証を行うことができる Issuer-Hiding AC の研究が進められている．

Issuer-Hiding AC では，検証者が受け入れ可能な発行元の公開鍵をポリシーに記載し（ポリシー作成），ユーザが実際の証明書の発行元がこのポリシー内に記載されていることを証明することで，発行元を秘匿したまま証明生成を可能とする．その中で，Bobolz らは，検証者が自身の信頼する発行元の公開鍵に署名を与えることで，効率的に Issuer-Hiding AC を実現するための一般的構成と，その一般的構成を Groth 署名 [4] を利用して，具体的に構成する方法を提案

した [5]．しかし，この手法は標準化が進んでいない Structure-Preserving 署名（群構造維持署名）に依存しており，現在標準化が進んでいたり，使われていたりする署名方式の多くが群構造維持署名でないことから実用化には遠い．また，Pointcheval-Sanders (PS) 署名 [6] を利用した Sanders と Traoré による Issuer-Hiding AC[7] や，BBS 署名を利用した Katz と Safranek による Issuer-Hiding AC[8] も提案されているが，前者は証明生成・検証の計算コストが非常に高いという課題があり，後者は安全性が Generic Group Model (GGM) [9] のもとでしか証明されていないため，標準モデルで安全性が証明できればその方が良い．

そこで本稿では，標準化が進んでいる BBS 署名を用いて Issuer-Hiding AC を構成できることを示す．具体的には，計算モデルとして標準モデルを仮定し，公開鍵を認証局などに登録する際に秘密鍵の所持証明も提出するという仮定である Registered Key Model を追加することで，Bobolz らの構成の安全性を BBS 署名の EUF-CMA 安全性に帰着させ，効率的な証明生成・検証が可能なシグマプロトコルを適用できる証明生成アルゴリズムを構成することで，Bobolz らの構成を BBS 署名で具体化可能であることを示す．

また，Issuer-Hiding AC を VC に適用するためのデータフォーマットや方法論についても，まだ十分に検討されていない．そこで，本稿では Issuer-Hiding AC を用いた VC や Verifiable Presentations（VP）のデータフォーマットを提案する．

本研究の貢献は以下の通りである．まず，BBS 署名を用いた Issuer-Hiding AC の構成を示し，その安全性を証明した．また，提案した Issuer-Hiding AC の方式と，PS 署名を用いた Sanders と Traorè による Issuer-Hiding AC[7]，BBS 署名を用いた Katz と Sefranek による Issuer-Hiding AC[8]，Groth 署名 [4] を用いた Bobolz らによる Issuer-Hiding AC[5] を計算コストやデータサイズの面で比較し，評価した．その結果，提案手法は既存方式と比較してポリシーの作成時の計算コストが最も低く，証明生成や証明検証の処理速度も高速であることを確認した．また，比較した 4 方式の中で唯一，計算モデルが標準モデルであることから，セキュリティ面でも優れていると考えられる．さらに，本稿では Issuer-Hiding AC を用いた Verifiable Credentials を構成するために，VC のデータモデル（JSON-LD[10] / Verifiable Credential Data Model v2.0[1]）に適合させた VC と Issuer-Hiding VP のデータフォーマットを提案した．

本稿の構成は以下の通りである．第 2 章では，本稿で用いる記法や暗号学的基礎知識について説明する．第 3 章では，Issuer-Hiding AC の定義と Bobolz ら，Sanders ら，Katz らの構成について説明する．第 4 章では，BBS 署名を用いた Issuer-Hiding AC の構成を示し，その安全性を証明する．第 5 章では，第 4 章で提案した構成と既存の Issuer-Hiding AC を計算コストやデータサイズの面で比較・評価する．第 6 章では，Issuer-Hiding AC を用いた Verifiable Credentials の構成例を示す．第 7 章では，本稿のまとめと今後の課題について述べる．

2 準備

2.1 双線形写像

G_1, G_2, G_T を位数 p の巡回群とする．$\mathbf{e} : G_1 \times G_2 \to G_T$ が以下の性質を満たすとき，$\mathbf{e}$ を双線形写像（ペアリング）という [11].

- 双線形性：任意の $a \in G_1, b \in G_2$ と $x, y \in \mathbb{Z}_p$ に対し，$\mathbf{e}(a^x, b^y) = \mathbf{e}(A, B)^{xy}$ が成り立つ.
- 非退化性：$g_1 \in G_1, g_2 \in G_2$ をそれぞれの群の生成元とするとき，$\mathbf{e}(g_1, g_2) \neq 1_{G_T}$ である.
- 計算可能性：任意の $a \in G_1, b \in G_2$ に対し，$\mathbf{e}(a, b)$ は効率的に計算可能である.

2.2 計算モデル

本稿における安全性証明および，本稿で取り扱う署名方式と Issuer-Hiding AC に関連する計算モデル（Standard Model, Generic Group Model, Algebraic Group Model）について述べる.

2.2.1 Generic Group Model (GGM)

Generic Group Model (GGM) は，Shoup [9] によって導入された，群演算をブラックボックスとして扱うモデルである．このモデルでは，群の具体的な表現（ビット列の構造）は隠蔽されており，敵対者は群演算をオラクルへの問い合わせを通じてのみ行うことができる．具体的には，ランダムな単射（エンコーディング関数）$\xi : \mathbb{Z}_p \to \{0,1\}^*$ を想定し，敵対者は群要素 g^x の代わりにそのハンドル $\xi(x)$ を受け取る．敵対者は，ハンドルのリストを用いて加算や逆元の計算をオラクルにリクエストすることで演算を行う．GGM は，群の数学的構造（特定の素因数分解の性質など）に依存した攻撃法が存在しないことを保証するために用いられる.

2.2.2 Standard Model（標準モデル）

Standard Model（標準モデル）とは，ランダムオラクルや GGM のような理想化された仮定を用いず，敵対者が群要素の具体的な表現（concrete representation）や構造にアクセスできる標準的な計算モデルである [9]．GGM のようなブラックボックス的な抽象化や，ランダムオラクルなどのヒューリスティックな仮定を用いず，敵対者は公開された群の具体的な表現（ビット列）に直接アクセスできるものとする．このモデルでの安全性証明は，離散対数問題（DL）や計算 Diffie-Hellman 問題（CDH）などの標準的な複雑性仮定への帰着によって行われるため，最も信頼性が高い.

2.2.3　Algebraic Group Model (AGM)

Algebraic Group Model (AGM) は，Fuchsbauer ら [12] によって提案された，標準モデルと GGM の中間に位置するモデルである．GGM とは異なり，敵対者は群要素の具体的なビット表現を見ることができる（標準モデルと同様）．しかし，AGM における敵対者 $\mathcal{A}$ は「代数的（Algebraic）」であると制限される．すなわち，$\mathcal{A}$ が群要素 Y を出力するとき，$\mathcal{A}$ は同時に入力された群要素 $\{X_1, \ldots, X_n\}$ に対する係数ベクトル $\vec{z} = (z_1, \ldots, z_n)$ を出力しなければならない（ここで $Y = \prod_{i=1}^{n} X_i^{z_i}$）．AGM は，敵対者の計算能力に対して GGM よりも現実的な制約を与えつつ，標準モデルでは証明が困難な効率的プロトコルの安全性証明を可能にする．

2.3　デジタル署名

デジタル署名 Σ は以下の 4 つのアルゴリズムから構成される．

- $pp \leftarrow \Sigma.\mathsf{ParGen}(1^\lambda)$：セキュリティパラメータ λ に対し，公開パラメータ pp を生成し，公開する．
- $(sk, pk) \leftarrow \Sigma.\mathsf{KGen}(pp)$：秘密鍵 sk と公開鍵 pk のペアを生成する．
- $\sigma \leftarrow \Sigma.\mathsf{Sign}(pp, sk, M)$：メッセージ M に対し，sk を用いて署名 σ を生成する．
- $b \leftarrow \Sigma.\mathsf{Verify}(pp, pk, M, \sigma)$：署名 σ がメッセージ M に対して有効か公開鍵 pk を用いて検証する．

デジタル署名 Σ が満たすべき要件として，Correctness と Unforgeability の 2 つがある．

定義 1 (Correctness)**.** デジタル署名 Σ が，Correctness を満たすとは，任意のセキュリティパラメータ λ，および任意のメッセージ M に対して，以下の確率が 1 であることを言う．

$$\Pr\begin{bmatrix} pp \leftarrow \Sigma.\mathsf{ParGen}(1^\lambda); \\ (sk, pk) \leftarrow \Sigma.\mathsf{KGen}(pp); \ : \ \Sigma.\mathsf{Verify}(pp, pk, M, \sigma) = 1 \\ \sigma \leftarrow \Sigma.\mathsf{Sign}(pp, sk, M) \end{bmatrix} = 1$$

すなわち，正規の手順で生成された鍵と署名に対し，検証アルゴリズムは常に受理（1）を出力しなければならない．

次に，デジタル署名の Unforgeability として，適応的選択メッセージ攻撃に対する存在的偽造不可能性（EUF-CMA）を定義する．

定義 2 (EUF-CMA 安全性)**.** デジタル署名 Σ が，適応的選択的メッセージ攻撃に対して存在的偽造不可能（EUF-CMA）であるとは，任意の確率的多項式時間（PPT）敵対者 $\mathcal{A}$ に対し，その有利性

$$\mathsf{Adv}^{\mathsf{EUF\text{-}CMA}}_{\Sigma,\mathcal{A}}(\lambda) := \Pr[\mathsf{Exp}^{\mathsf{EUF\text{-}CMA}}_{\Sigma,\mathcal{A}}(\lambda) = 1]$$

が，セキュリティパラメータ λ において無視可能（negligible）であることを言う [13]．ここで，実験 $\mathsf{Exp}^{\mathsf{EUF\text{-}CMA}}_{\Sigma,\mathcal{A}}(\lambda)$ は図 1 のように定義される．

$\underline{\mathsf{Exp}^{\mathsf{EUF\text{-}CMA}}_{\Sigma,\mathcal{A}}(\lambda)}$

$pp \xleftarrow{\$} \Sigma.\mathsf{ParGen}(1^\lambda)$

$(sk, pk) \xleftarrow{\$} \Sigma.\mathsf{KGen}(pp)$

$\mathcal{Q} \leftarrow \emptyset$

$(M^*, \sigma^*) \leftarrow \mathcal{A}^{\mathcal{O}_{\mathsf{Sign}}}(pp, pk)$

If $M^* \in \mathcal{Q}$, return 0

If $\Sigma.\mathsf{Verify}(pp, pk, M^*, \sigma^*) = 1$, return 1

Return 0

$\underline{\mathcal{O}_{\mathsf{Sign}}(M)}$

$\sigma \xleftarrow{\$} \Sigma.\mathsf{Sign}(pp, sk, M)$

$\mathcal{Q} \leftarrow \mathcal{Q} \cup \{M\}$

return σ

図 1　EUF-CMA 実験

2.4　コミットメント付きデジタル署名

デジタル署名の 4 つのアルゴリズムに加えて，以下のアルゴリズムを備えたものをコミットメント付きデジタル署名 Σ_C と定義する．

- $C \leftarrow \Sigma_C.\mathsf{Commit}(pp, M)$：メッセージ M に対するコミットメント C を生成する．
- $\sigma \leftarrow \Sigma_C.\mathsf{CSign}(pp, sk, C)$：コミットメント C に対し，sk を用いて署名 σ を生成する．
- $b \leftarrow \Sigma_C.\mathsf{CVerify}(pp, pk, C, \sigma)$：署名 σ がコミットメント C に対して有効か公開鍵 pk を用いて検証する．

コミットメント付きデジタル署名 Σ_C が満たすべき要件も Correctness と Unforgeability の 2 つがある．

定義 3 (Σ_C の Correctness)**.** コミットメント付きデジタル署名 Σ_C が，Correctness を満たすとは，任意のセキュリティパラメータ λ，および任意のメッセージ M に対して，以下の確率が 1 であることを言う．

$$\Pr\begin{bmatrix} pp \leftarrow \Sigma.\mathsf{ParGen}(1^\lambda); \\ (sk, pk) \leftarrow \Sigma.\mathsf{KGen}(pp); \\ C \leftarrow \Sigma_C.Commit(pp, M); \\ \sigma \leftarrow \Sigma_C.\mathsf{CSign}(pp, sk, C) \end{bmatrix} : \Sigma_C.\mathsf{CVerify}(pp, pk, C, \sigma) = 1 \Bigg] = 1$$

なお，任意の M について $\mathsf{Sign}(sk, M) = \mathsf{CSign}(sk, \mathsf{Commit}(M))$ が成立するものとする．

本稿において，コミットメント付きデジタル署名は公開鍵への署名にのみ利用される（3 章参照）．後述する Certified Key Settings（2.15 節参照）では，登録された公開鍵に対応する秘密鍵を抽出可能であるため，3.1.3 節の安全性証明においてコミットメント特有の性質に依存する必要がない．したがって本稿では，通常のデジタル署名としての EUF-CMA 安全性のみを仮定する．すなわち，コミットメントではなくメッセージを入力に取る (CSign ではなく Sign を使った) 署名オラクル $\mathcal{O}_{\mathrm{sign}}$ へのアクセスを持つ攻撃者が，(コミットメントではなく) メッセージに

関する偽造ができないことのみを要求する．

2.5　BBS 署名

BBS 署名は，元々は Boneh らによって提案された [14] グループ署名の署名方式であり，Au らによって複数のメッセージへの署名が可能な形式へと拡張された [15]．この際，安全性証明のために署名値に追加の項を加え 1 つの群要素と 2 つのスカラー値からなることから BBS+ 署名と呼ばれている．その後，Tessaro と Zhu により署名値が 1 つの群要素と 1 つのスカラー値からなる場合でも安全であることが証明されている [2]．本稿では，Tessaro と Zhu が証明した署名値が 1 つの群要素と 1 つのスカラー値からなる場合の BBS 署名を利用する．

BBS 署名は，以下のアルゴリズムから構成される．

- $\mathsf{BBS.ParGen}(1^\lambda)$：位数 p の双線形群を構成する公開パラメータ $pp = (G_1, G_2, G_T, \mathbf{e}, p, g_1, g_2, \{h_i\}_{i=1}^{l})$ を生成する．ここで，$g_1 \in G_1, g_2 \in G_2$ はそれぞれの生成元，$h_i \xleftarrow{\$} G_1$ はランダムに選ばれた要素, l はメッセージ数である．公開パラメータ pp を返す．
- $\mathsf{BBS.KGen}(pp)$：ランダムに $x \xleftarrow{\$} \mathbb{Z}_p$ を選び，秘密鍵 $sk = x$，公開鍵 $pk = g_2^x$ を生成する．(sk, pk) を返す．
- $\mathsf{BBS.Sign}(pp, sk, \mathbf{m})$：メッセージ $\mathbf{m} = (m_1, m_2, \ldots, m_l)$ に対し，$C(\mathbf{m}) = g_1 \prod_{i=1}^{l} h_i^{m_i}$ を計算する．ランダムに $e \xleftarrow{\$} \mathbb{Z}_p$ を選び，以下の計算を行う．

$$A = C(\mathbf{m})^{\frac{1}{sk+e}}$$

  署名 $\sigma = (A, e)$ を返す．
- $\mathsf{BBS.Verify}(pp, pk, \mathbf{m}, \sigma)$：$C(\mathbf{m})$ を計算し，以下のペアリングの等式が成り立つか確認する．

$$\mathbf{e}(A, pk \cdot g_2^e) \stackrel{?}{=} \mathbf{e}(C(\mathbf{m}), g_2)$$

本稿ではこれらのアルゴリズムを BBS_I と呼ぶ．

BBS 署名は標準モデルのもとで次に定義する Strong EUF-CMA (SUF) を満たすことが証明されている [2] ほか，AGM のもとでよりタイトに SUF を満たすことが証明されている．

定義 4 (SUF 安全性)**.** デジタル署名 Σ が，適応的選択メッセージ攻撃に対して強存在的偽造不可能（SUF ）であるとは，任意の確率的多項式時間（PPT）敵対者 $\mathcal{A}$ に対し，その有利性

$$\mathsf{Adv}^{\mathsf{SUF}}_{\Sigma,\mathcal{A}}(\lambda) := \Pr[\mathsf{Exp}^{\mathsf{SUF}}_{\Sigma,\mathcal{A}}(\lambda) = 1]$$

が，セキュリティパラメータ λ において無視可能（negligible）であることを言う．実験 $\mathsf{Exp}^{\mathsf{SUF}}_{\Sigma,\mathcal{A}}(\lambda)$ は図 2 のように定義される．

定義 2 と 4 より，SUF 安全であれば EUF-CMA 安全でもある．

$\underline{\mathsf{Exp}^{\mathsf{SUF}}_{\Sigma,\mathcal{A}}(\lambda)}$

$pp \xleftarrow{\$} \Sigma.\mathsf{ParGen}(1^\lambda)$

$(sk, pk) \xleftarrow{\$} \Sigma.\mathsf{KGen}(pp)$

$\mathcal{Q} \leftarrow \emptyset$

$(M^*, \sigma^*) \leftarrow \mathcal{A}^{\mathcal{O}_{\mathsf{Sign}}}(pp, pk)$

If $(M^*, \sigma^*) \in \mathcal{Q}$, return 0

If $\Sigma.\mathsf{Verify}(pp, pk, M^*, \sigma^*) = 1$, return 1

Return 0

$\underline{\mathcal{O}_{\mathsf{Sign}}(M)}$

$\sigma \leftarrow \Sigma.\mathsf{Sign}(pp, sk, M)$

$\mathcal{Q} \leftarrow \mathcal{Q} \cup \{(M, \sigma)\}$

Return σ

図 2　SUF 実験

また，BBS_I の G_1 と G_2 を交換し，署名対象メッセージを一つのみ取るようにし，$C(\mathbf{m})$ の計算に用いる基底 h_1 の代わりに BBS_I の生成元 g_2 を利用し，以下のアルゴリズムを用いるものを BBS_V と呼ぶ.

- $\mathsf{ParGen}(1^\lambda) \to pp = (G_1, G_2, G_T, \mathbf{e}, p, \bar{g}_1, \bar{g}_2, g_2)$
- $\mathsf{KGen}(pp) \to (sk = x, pk = \bar{g}_1^x)$
- $\mathsf{Sign}(pp, sk, m) \to (A = (\bar{g}_2 \cdot g_2^m)^{\frac{1}{sk+e}}, e)$
- $\mathsf{Verify}(pp, pk, m, \sigma) \to [\mathbf{e}(pk \cdot \bar{g}_1^e) \stackrel{?}{=} \mathbf{e}(\bar{g}_1, \bar{g}_2 \cdot g_2^m)]$
- $\mathsf{Commit}(pp, m) \to \bar{g}_2 \cdot g_2^m$,
- $\mathsf{CSign}(pp, sk, C) \to (A = C^{\frac{1}{sk+e}}, e)$,
- $\mathsf{CVerify}(pp, pk, C, \sigma) \to [\mathbf{e}(pk \cdot \bar{g}_1^e, A) \stackrel{?}{=} \mathbf{e}(\bar{g}_1, C)]$

BBS_V は，コミットメントに関わる部分を除けば BBS_I と同じ構造を持つため，[2] の結果に基づいて EUF-CMA 安全であると言える (本稿ではコミットメント部分を無視した EUF-CMA 安全性のみを扱うことに注意).

2.6　PS 署名

PS 署名は，以下のアルゴリズムから構成される [6].

- $\mathsf{PS.ParGen}(1^\lambda)$：位数 p の双線形群を構成する公開パラメータ $pp = (G_1, G_2, G_T, \mathbf{e}, p, g_1, g_2)$ を生成する．ここで，$g_1 \in G_1, g_2 \in G_2$ はそれぞれの生成元である．公開パラメータ pp を返す.
- $\mathsf{PS.KGen}(pp)$：ランダムに $x, y_i \xleftarrow{\$} \mathbb{Z}_p^{l+1}$ を選び，秘密鍵 $sk = (x, y_i)$，公開鍵 $pk = (X = g_2^x, Y_i = g_2^{y_i})$ を生成する．ここで，l はメッセージ数である．(sk, pk) を返す.
- $\mathsf{PS.Sign}(pp, sk, \mathbf{m})$：メッセージ $\mathbf{m} = (m_1, m_2, \ldots, m_l)$ とランダムな値 $r \xleftarrow{\$} \mathbb{Z}_p$ に対し，

以下の計算を行う．

$$\sigma_1 = g_1^r$$
$$\sigma_2 = g_1^{r\cdot(x+\sum_{i=1}^{l} y_i \cdot m_i)}$$

を計算する．署名 $\sigma = (\sigma_1, \sigma_2)$ を返す．

- $\mathsf{PS.Verify}(pp, pk, \sigma, \mathbf{m})$：以下のペアリングの等式が成り立つか確認する．

$$\mathbf{e}(\sigma_1, X \cdot \prod_{i=1}^{l} Y_i^{m_i}) \stackrel{?}{=} \mathbf{e}(\sigma_2, g_2)$$

PS 署名は EUF-CMA 安全性を満たすことが証明されている．

2.7 Groth 署名

Groth 署名は，以下のアルゴリズムから構成される [4].

- $\mathsf{Groth}_1.\mathsf{ParGen}$: 公開パラメータ $pp := (G_1, G_2, G_T, p, \mathbf{e}, G_1, g_2, y)$ を出力する．ただし，$g_1 \in G_1, g_2 \in G_2$ はそれぞれの群の生成元，$y \stackrel{\$}{\leftarrow} G_1$ はランダムに選ばれた要素である．
- $\mathsf{Groth}_1.\mathsf{KeyGen}(pp)$: 秘密鍵を $sk \stackrel{\$}{\leftarrow} \mathbb{Z}_p^*$ で求め，公開鍵を $pk := g_2^{sk}$ とする．鍵ペア (sk, pk) を返す．
- $\mathsf{Groth}_1.\mathsf{Sign}(sk, M)$: ランダムな $r \stackrel{\$}{\leftarrow} \mathbb{Z}_p^*$ を用意し，署名 $\sigma := (R_2, S_1, T_1)$ を以下のように求める．

$$\sigma = (R_2, S_1, T_1) := (g_2^r, (y \cdot g_1^{sk})^{1/r}, (y^{sk} \cdot M)^{1/r})$$

  σ を返す．
- $\mathsf{Groth}_1.\mathsf{Rand}(pp, \sigma)$: ランダムな $r' \stackrel{\$}{\leftarrow} \mathbb{Z}_p^*$ を用意し，ランダム化された署名 $\sigma' := (R_2', S_1', T_1')$ を以下のように求める．

$$\sigma' = (R_2', S_1', T_1') := (R_2^{r'}, S_1^{1/r'}, T_1^{1/r'})$$

  σ' を返す．
- $\mathsf{Groth}_1.\mathsf{Verify}(pk, M, \sigma)$: 以下を計算し，成り立てば 1 を，そうでなければ 0 を返す．

$$\mathbf{e}(S_1, R_2) \stackrel{?}{=} \mathbf{e}(y, g_2) \cdot \mathbf{e}(g_1, pk)$$
$$\mathbf{e}(T_1, R_2) \stackrel{?}{=} \mathbf{e}(y, pk) \cdot \mathbf{e}(M, g_2)$$

2.8 ゼロ知識証明 (Zero-Knowledge Proofs)

ゼロ知識証明は，Goldwasser, Micali, Rackoff [16] によって導入された概念であり，ある主張の正しさを「その正しさ以外の情報を一切漏らすことなく」証明する手法である．

定義 5 (ゼロ知識証明)**.** NP 言語 L とその証拠関係 R を考える．$(x, w) \in R$ である入力 x と証拠 w を持つ証明者 P と，入力 x を持つ検証者 V の間の対話型プロトコル Π が，言語 L に対するゼロ知識証明システムであるとは，以下の 3 つの性質を満たすことを言う．

- **完全性 (Completeness):** $x \in L$ ならば，正しい証拠 w を持つ P は V を圧倒的な確率で受理させることができる．
- **健全性 (Soundness):** $x \notin L$ ならば，いかなる悪意ある証明者 P^* も，V を無視できない確率で受理させることはできない．
- **ゼロ知識性 (Zero-Knowledge):** $x \in L$ ならば，任意の（悪意ある）検証者 V^* に対し，P との対話から得られる情報（ビュー）と計算的に識別不可能な出力を生成する確率的多項式時間アルゴリズム（シミュレータ）$\mathcal{S}$ が存在する．

2.9　シグマプロトコル

本稿で用いるゼロ知識証明の多くは，シグマプロトコル（Σ-protocol）[17], [18] と呼ばれるクラスに属する．シグマプロトコルは，証明者（Prover）と検証者（Verifier）の間で行われる 3 動作（3-move）の対話型プロトコルであり，その通信フロー（Commitment, Challenge, Response）の形状がギリシャ文字の Σ に似ていることから名付けられた．

Damgård [17] の定義に従い，関係 $R \subseteq \{0,1\}^* \times \{0,1\}^*$ に対するシグマプロトコルを以下のように定義する．入力 x と証拠 w　$((x, w) \in R)$ を持つ証明者 P と，入力 x を持つ検証者 V は以下の対話を行う．

- Commitment: P は初期メッセージ（コミットメント）a を計算し，V に送信する．
- Challenge: V はランダムなチャレンジ $e \leftarrow \mathbb{Z}_q$ を選択し，P に送信する．
- Response: P は (x, w, a, e) に基づいて応答 z を計算し，V に送信する．
- Verification: V は受理判定アルゴリズムを実行し，(x, a, e, z) に基づいて受理（Accept）か拒否（Reject）を出力する．

シグマプロトコルは以下の 3 つの性質を満たす必要がある．

- Completeness: 正直な証明者が正しい手順に従えば，検証者は確率 1 で受理する．
- Special Soundness: 同一の入力 x とコミットメント a に対し，異なるチャレンジ $e \neq e'$ に対応する 2 つの受理可能なトランスクリプト (a, e, z) と (a, e', z') が存在すれば，そこから証拠 w を効率的に計算（抽出）できる．
- Special Honest-Verifier Zero-Knowledge, SHVZK: 任意のチャレンジ e に対し，正しい分布に従うトランスクリプト (a, e, z) を，証拠 w を知らずに生成できるシミュレータが存在する．

2.10　非対話ゼロ知識証明 (NIZK)

実際の暗号プロトコル，特にブロックチェーンや匿名認証においては，通信回数を削減し，証明をデータとして保存・転送可能にするために，対話を必要としない非対話ゼロ知識証明（Non-Interactive Zero-Knowledge, NIZK）が用いられる．NIZK は，Blum, Feldman, Micali [19] により，共通参照文字列（Common Reference String, CRS）モデルにおいて初めて定義された．

定義 6 (NIZK 証明システム)**.** 関係 R に対する NIZK 証明システムは，以下の 3 つのアルゴリズム (Setup, Prove, Verify) から構成される．

- $crs \leftarrow \mathsf{Setup}(1^\lambda)$: セキュリティパラメータ λ を入力とし，共通参照文字列 crs を出力する．
- $\pi \leftarrow \mathsf{Prove}(crs, x, w)$: crs，文 x，証拠 w を入力とし，証明 π を出力する．
- $b \leftarrow \mathsf{Verify}(crs, x, \pi)$: crs, x, π を入力とし，受理 (1) か拒否 (0) を出力する．

NIZK は，前述の完全性と健全性に加え，以下のゼロ知識性を満たす．

- ゼロ知識性: シミュレータ $\mathcal{S} = (\mathcal{S}_1, \mathcal{S}_2)$ が存在し，$\mathcal{S}_1$ が生成するシミュレートされた crs とトラップドア τ，および $\mathcal{S}_2$ が証拠 w なしで生成する証明 π が，本物のプロトコル実行時の (crs, π) と識別不可能である．

なお，前小節で述べたシグマプロトコルに対して Fiat-Shamir 変換 [20] を適用することで，ランダムオラクルモデル（ROM）における NIZK を構成することが可能である．本稿で用いる署名方式や認証プロトコルの一部はこの構成に従う．

2.11　知識証明（Proofs of Knowledge）と抽出可能性 (Extractability)

ゼロ知識証明システムにおいて，証明者 P が単に検証者 V に受理させるだけでなく，実際に証拠 w を所持していることを保証する性質を「知識の証明（Proof of Knowledge）」と呼ぶ．この概念は，Bellare と Goldreich [21] によって形式化された「抽出可能性（Extractability）」によって定義される．

定義 7 (抽出可能性 / 知識証明)**.** 対話型証明システム (P, V) が関係 R に対する知識の証明（Proof of Knowledge）であるとは，ある関数 $\kappa(\cdot)$（知識誤差，knowledge error）に対し，以下の性質を満たす確率的多項式時間アルゴリズム E（知識抽出機，Knowledge Extractor）が存在することを言う [21].

任意の証明者 P^* と任意の入力 x に対し，P^* が V を確率 $\varepsilon(x) > \kappa(x)$ で受理させるならば，抽出機 E は P^* へのブラックボックスアクセス（およびその内部状態の巻き戻しが可能であること）を利用して，$(x, w) \in R$ なる証拠 w を，$\varepsilon(x)$ に反比例する期待多項式時間で出力（抽出）できる．

すなわち，検証者を納得させられる証明者からは，必ず証拠 w を抽出可能である．

なお，前述のシグマプロトコルにおいては，Special Soundness により，2 つの異なるチャレンジに対する応答から証拠を計算できるため，この抽出可能性を自然に満たす．

2.12 Simulation-Sound Extractable NIZK

Bernhard らは NIZK (Non-Interactive Zero-Knowledge Proof) が Simulation-Sound Extractable であるための条件について提示し，証明している [22]．[22, Theorem 1] では，セキュリティパラメータのチャレンジ空間が指数サイズなシグマプロトコルが Special Soundness と Special Honest-Verifier Zero-Knowledge を満たしているときに，強 Fiat-Shamir 変換を用いて得られる NIZK は Simulation-Sound Extractable であるとしている．強 Fiat-Shamir 変換の定義は [22, Definition 2] に記述されている．

2.13 匿名認証 (Anonymous Credentials)

匿名認証システム（Anonymous Credential System）は，Chaum [23] によって提唱されたプライバシー保護技術の概念であり，ユーザが自身のアイデンティティを明かすことなく，自身が正当な資格（Credential）を有していることを証明可能にする．Camenisch らは [24]，Chaum の概念をもとに Anonymous Credentials (AC) の定式化を行なった．AC は以下の主要なプロトコルから構成される．

- Setup: システム全体のパラメータを生成する．
- KeyGen: 発行者（Issuer）は秘密鍵 isk と公開鍵 ipk を生成する．（必要に応じてユーザも鍵ペアを生成する）
- Issue: ユーザと発行者の間の対話プロトコル．ユーザは自身の属性（Attributes）x を発行者に送信し，発行者は isk を用いて属性に対する認証情報（Credential）σ を発行する．この際，ブラインド署名等の技術により，発行者はユーザの属性を知らずに発行することも可能である．
- Show (または Prove): ユーザと検証者（Verifier）の間の対話プロトコル．ユーザは発行者の公開鍵 ipk，属性 x，および認証情報 σ を用いて，正当な資格を保持していることを証明する．この際，ゼロ知識証明を用いることで，属性値の一部のみを選択的に開示（Selective Disclosure）したり，属性値そのものを隠したまま述語（年齢が 18 歳以上である等）の成立を証明したりすることができる．
- Verify: 検証者は，Show プロトコルの出力（証明 π）を検証し，受理か拒否を出力する．

AC システムには，Correctness の他に主に以下の安全性が求められる．

偽造不可能性 (Unforgeability): 正当な発行者との Issue プロトコルを経ずに，有効な認証情報 σ を生成することはできない（基礎となる署名方式の安全性に帰着される）．

連結不可能性 (Unlinkability): 同一の認証情報 σ を用いて複数回の Show プロトコルを実行しても，検証者（および発行者）はそれらが同一のユーザによるものであることを識別できない．

2.14 Issuer-Hiding AC の概要

ここでは，Bobolz らによる Issuer-Hiding AC の一般的構成 [5] をベースに Issuer-Hiding AC の概要を示す．Bobolz らは，複数の発行元からなる集合の中に実際の証明書の発行元が含まれることを示すことで，実際の発行元を特定せずとも証明検証時に検証可能となるようにした．具体的には，検証者が受け入れる発行元の公開鍵に対して署名を与え，ポリシーを発行する．このポリシーを利用して，ユーザは実際の発行元の公開鍵を秘匿しつつ，検証者の署名がついていることを証明することで，発行元の秘匿と属性の検証を両立させている．

2.14.1 シンタックス

Issuer-Hiding AC のシンタックスを示す．

- $pp \leftarrow \mathsf{ParGen}(1^\lambda)$：公開パラメータを生成する．
- $(isk, ipk) \leftarrow \mathsf{IKGen}(pp)$：発行元の鍵生成を行う．
- $cred \leftarrow \mathsf{Issue}(isk, \mathbf{m})$：発行元が属性 $\mathbf{m}$ に対し，証明書 $cred$ を生成する．
- $b \leftarrow \mathsf{VfCred}(cred, \mathbf{m}, ipk)$：証明書 $cred$ が有効か，検証する．
- $pol \leftarrow \mathsf{PresPolicy}(\{ipk_i\})$：検証者は信頼する発行元の公開鍵集合 $\{ipk_i\}$ を用い，ポリシー pol を生成する．
- $b \leftarrow \mathsf{VfPolicy}(pol, \{ipk_i\})$：ポリシー pol が有効か，検証する．
- $pt \leftarrow \mathsf{Present}(cred, \mathbf{m}, ipk, \phi, pol, \mathsf{ctx})$：ユーザは，ポリシー pol に基づき，証明と提示トークン pt を生成する．
- $b \leftarrow \mathsf{Verify}(pt, \phi, pol, \mathsf{ctx})$：提示トークン pt が有効か，検証する．

2.14.2 安全性

Bobolz らは安全性として以下の 3 つの要件を提示している．

- Correctness：全てのフェーズでプロトコル仕様に従う場合，ユーザが計算した提示トークンは検証者によって受け入れられる．
- Unforgeability：次のいずれも受け取っていない攻撃者が有効な提示を生成することが困難である．
  - 認証情報 ϕ を満たす属性に対し，受け入れ可能な発行元のいずれかから発行された証明書
  - 同じ $(\mathsf{ctx}, \phi, pol)$ に対する提示
- Unlinkability：攻撃者が 2 つのユーザの操作を関連づけることができない．この要件は，

攻撃者が検証者，発行元，およびユーザの資格情報を完全に制御している場合でも成立する必要がある．

2.15　登録鍵モデル (Registered Key Model)

本稿の Issuer-Hiding AC システムでは，検証者が複数の発行元の公開鍵に対して署名を行う状況を想定する．この際，悪意ある攻撃者が他の発行者の公開鍵に依存した不正な鍵を生成する攻撃（Rogue Key Attack）を排除するため，Ristenpart と Yilek[25] によって形式化された登録鍵モデル（Registered Key Model）を採用する．Ristenpart らは，攻撃者が自身の公開鍵を任意に選択可能な状況（Plain Public-Key Model）では，多くのマルチパーティ署名スキームが Rogue Key Attack に対して脆弱であることを指摘している．これを防ぐために，発行者（または攻撃者）が認証局に対して鍵登録プロトコルを実行し，成功した場合にのみその公開鍵がシステム内で有効になると定義される．

定義 8 (Registered Key Model)**.** 登録鍵モデルにおいて，公開鍵の登録は，登録プロトコル $(\mathsf{RegP}, \mathsf{RegV})$ を通じて行われる．敵対者 $\mathcal{A}$ は，自身の公開鍵 pk を登録するために RegP を実行し，RegV が成功した場合にのみ，pk は有効な鍵として登録される．

本稿では，RegP として Proof of Possession（PoP）に基づく登録プロトコルを考える．すなわち，$\mathcal{A}$ は (pk, π) を提出し，π は公開鍵 pk に対応する秘密鍵 sk の所持証明である．オラクル $\mathcal{O}_{\mathsf{Reg}}$ は，$\mathsf{VerifyPoP}(pk, \pi) = 1$ が成立する場合にのみ，pk を有効な鍵として登録する．

Bellare と Neven [26] により議論されている KOSK（Knowledge of Secret Key）モデルでは，敵対者は鍵登録時に秘密鍵そのものを提出すると仮定されるが，これは既存の PKI においては現実的ではない．Ristenpart と Yilek [25] は，特定のマルチパーティ署名スキームに対して，ドメイン分離された署名等を用いた適切な PoP プロトコルの下で，KOSK 仮定に基づく安全性証明と同様の安全性が達成できることを示している．

本稿ではこの知見に基づき，登録時に用いられる証明 π が知識証明としての抽出可能性を満たすことを仮定する．これにより，安全性証明において登録された π から対応する秘密鍵 sk を抽出可能となり，基礎となる署名方式の安全性（EUF-CMA 等）への帰着が可能となる．

3 Issuer-Hiding AC

3.1 Issuer-Hiding AC の一般的構成

Bobolz らは利用する署名方式を群構造維持署名であるということ以外指定しない一般的構成を提示した [5]．本節ではこの一般的構成を説明した後，群構造維持署名でない署名方式でも使えるように一般化し，安全性を証明する．

3.1.1 Issuer-Hiding AC の一般的な構成

Σ_I を発行元によるデジタル署名のアルゴリズムとし，$\Sigma_I = (\mathsf{ParGen}, \Sigma_I.\mathsf{KGen}, \Sigma_I.\mathsf{Sign}, \Sigma_I.\mathsf{Verify})$ とする．また，Σ_V を検証者が用いるコミットメント付きデジタル署名のアルゴリズムとし，$\Sigma_V = (\mathsf{ParGen}, \Sigma_V.\mathsf{KGen}, \Sigma_V.\mathsf{Sign}, \Sigma_V.\mathsf{Verify}$ とする．なお，ここで ParGen は公開パラメータの生成，KGen は鍵ペア生成，Sign は署名生成，Verify は署名検証のアルゴリズムを指す．なお，Σ_V のメッセージ空間は Σ_I の公開鍵空間と一致する．

2.14.1 節で示したシンタックスを用いて Issuer-Hiding AC の一般的構成を図 3 に示す．

$\mathsf{ParGen}(1^\lambda)$: Return $pp \leftarrow \Sigma.\mathsf{ParGen}(1^\lambda)$.
$\mathsf{IKGen}(pp)$: Return $(isk, ipk) \xleftarrow{\$} \Sigma_I.\mathsf{KGen}(pp)$.
$\mathsf{Issue}(isk, \mathbf{m})$: Return $cred \xleftarrow{\$} \Sigma_I.\mathsf{Sign}(pp, isk, \mathbf{m})$.
$\mathsf{VfCred}(cred, \mathbf{m}, ipk)$: Return $\Sigma_I.\mathsf{Verify}(pp, ipk, cred, \mathbf{m})$
$\mathsf{VKGen}(pp)$: Return $(vsk, vpk) \xleftarrow{\$} \Sigma_V.\mathsf{KGen}(pp)$.
$\mathsf{PresPolicy}(\{ipk_i\})$: $\sigma_i \leftarrow \Sigma_V.\mathsf{Sign}(pp, vsk, ipk_i)$. Return $pol = (vpk, \{ipk_i, \sigma_i\})$.
$\mathsf{VfPolicy}(pol, \{ipk_i\})$: Parse pol as $(vpk, \{ipk_i, \sigma_i\})$. Return 1 if and only if $\Sigma_V.\mathsf{CVerify}(pp, vpk, \sigma_i, ipk_i) = 1$ for all i. Otherwise return 0.
$\mathsf{Present}(pp, cred, ipk, \mathbf{m}, \phi, pol, \mathsf{ctx})$: Parse pol as $(vpk, \{ipk_i, \sigma_i\})$. Set j such that $ipk = ipk_j$. Return a presentation token pt as follows:

$$\begin{aligned} pt \leftarrow \mathrm{NIZK}[(cred_j, ipk_j, cred, \mathbf{m}) :& \\ &\Sigma_I.\mathsf{Verify}(pp, ipk_j, cred, \mathbf{m}) = 1 \wedge \\ &\Sigma_V.\mathsf{Verify}(pp, vpk, \sigma_j, ipk_j) = 1 \wedge \\ &\phi(\mathbf{m}) = 1](pol, \phi, \mathsf{ctx}) \end{aligned}$$

$\mathsf{Verify}(pt, pol, \phi, \mathsf{ctx})$: Return 1 if and only if pt verifies correctly. Otherwise return 0.

図 3　Issuer-Hiding AC の一般的構成

Bobolz らは 2.14.2 で示した安全性要件を定義 9, 10 のように定義し，Issuer-Hiding AC の一般的構成がその定義を満たすことを示している．なお，Correctness は自明なため，Unforgeability と Unlinkability について Bobolz らの定義を示す．

まず，定義 9 に Unforgeability の定義を示す．この定義において注意すべきは，チャレンジ

ポリシー pol^* は honest な発行元の鍵のみを含む一方で，攻撃者は提示オラクル $\mathcal{O}_{\mathrm{present}}$ に対して，任意の発行元の公開鍵の集合に基づく提示を要求できるという点である．これは，悪意のある発行元も含まれるように設計されている．

定義 9. Issuer-Hiding AC が Unforgeability を満たすのは，任意の PPT 攻撃者 Adv および，任意の発行者数 n_I に対して，無視可能な関数（negligible function）negl が存在し，以下の条件を満たす場合である．

$$\Pr\left[\mathbf{Exp}_{\mathsf{Adv}}^{\mathsf{Unforgeability}}(\lambda, n_I) = 1\right] \leq \mathsf{negl}(\lambda)$$

ただし，$\mathbf{Exp}_{\mathsf{Adv}}^{\mathsf{Unforgeability}}(\lambda, n_I)$ は図 4 で定義される．

$\mathbf{Exp}_{\mathsf{Adv}}^{\mathsf{Unforgeability}}(\lambda, n_I)$:
  $pp \xleftarrow{\$} \mathsf{ParGen}(1^\lambda)$
  $Q_{\mathsf{issue}} \leftarrow \emptyset, \quad Q_{\mathsf{present}} \leftarrow \emptyset, \quad Q_{\mathsf{reveal}} \leftarrow \emptyset$
  $(isk_i, ipk_i) \leftarrow \mathsf{IKGen}(pp) \quad \text{for } i \in [n_I]$
  $(I^*, st) \leftarrow \mathsf{Adv}^{\mathcal{O}_{\mathsf{issue}}, \mathcal{O}_{\mathsf{present}}, \mathcal{O}_{\mathsf{reveal}}}(pp, \{ipk_i\}_{i=1}^{n_I})$
  $pol^* \leftarrow \mathsf{IssueList}(I^*)$
  $(pt^*, \phi^*, \mathsf{ctx}^*) \leftarrow \mathsf{Adv}^{\mathcal{O}_{\mathsf{issue}}, \mathcal{O}_{\mathsf{present}}, \mathcal{O}_{\mathsf{reveal}}}(st, pol^*)$
  where the oracles are defined as follows:
    $\mathcal{O}_{\mathsf{issue}}(i_j, \mathbf{m}_j)$
      $cred_j \leftarrow \mathsf{Issue}(isk_{i_j}, \mathbf{m}_j)$
      add $(i_j, \mathbf{m}_j)$ to Q_{issue}
    $\mathcal{O}_{\mathsf{present}}(j, pol, \phi, \mathsf{ctx})$
      add $(pol, \phi, \mathsf{ctx})$ to Q_{present}
      return $pt \leftarrow \mathsf{Present}(cred_j, ipk_{i_j}, \mathbf{m}_j, \phi, pol, \mathsf{ctx})$
    $\mathcal{O}_{\mathsf{reveal}}(j)$
      add $(i_j, \mathbf{m}_j)$ to Q_{reveal}
      return $cred_j$
  return 1 if:
    $I^* \subseteq \{ipk_i\}_{i=1}^{n_I}$
    $\mathsf{Verify}(pt^*, pol^*, \phi^*, \mathsf{ctx}^*) = 1$
    $(pol^*, \phi^*, \mathsf{ctx}^*) \notin Q_{\mathsf{present}}$
    $\nexists (i_j, \mathbf{m}_j) \in Q_{\mathsf{reveal}}$ s.t. $\phi^*(\mathbf{m}_j) = 1$ and $ipk_{i_j} \in I^*$
  return 0

図 4　Unforgeability の実験

次に，定義 10 に Unlinkability の定義を示す．

定義 10. Issuer-Hiding AC システムが Unlinkability を満たすのは，任意の PPT 攻撃者 Adv に対して，無視可能な関数（negligible function）negl が存在し，以下の条件を満たす場合である．

$$\left|\Pr\left[\mathbf{Exp}_{\mathsf{Adv}}^{\mathsf{Unlinkability}}(\lambda)=1\right]-\frac{1}{2}\right|\leq \mathsf{negl}(\lambda)$$

ただし，$\mathbf{Exp}_{\mathsf{Adv}}^{\mathsf{Unlinkability}}(\lambda)$ は図 5 で定義される．

$\mathbf{Exp}_{\mathsf{Adv}}^{\mathsf{Unlinkability}}(\lambda)$:
$b \xleftarrow{\$} \{0,1\}, pp \xleftarrow{\$} \mathsf{ParGen}(1^\lambda)$
$(\{cred_k, \mathbf{m}_k, i_k\}_{k=0}^1, pol, \phi, cred_b, \{ipk_i\}, \mathsf{ctx}, st) \xleftarrow{\$} \mathsf{Adv}(pp)$
$pt^* \xleftarrow{\$} \mathsf{Present}(\{ipk_i\}, cred_b, \mathbf{m}_b, \phi, pol, \mathsf{ctx})$
$b^* \xleftarrow{\$} \mathsf{Adv}(pt^*, st)$
return 1 if and only if:
$b = b^*$
$\mathsf{VfCred}(cred_k, \mathbf{m}_k, ipk_{i_k}) = 1$ for $k = \{0,1\}$,
$\phi(\mathbf{m}_k) = 1$ for $k = \{0,1\}$, and $\mathsf{VfList}(pol, \{ipk_i\}) = 1$
return $b' \xleftarrow{\$} \{0,1\}$

図 5 Unlinkability の実験

3.1.2 群構造維持署名を用いない Issuer-Hiding AC の構成

Σ_V を検証者が用いるコミットメント付きデジタル署名のアルゴリズムとし，$\Sigma_V = (\mathsf{ParGen}, \Sigma_V.\mathsf{KGen}, \Sigma_V.\mathsf{Sign}, \Sigma_V.\mathsf{CSign}, \Sigma_V.\mathsf{Verify}, \Sigma_V.\mathsf{CVerify})$ とする．なお，ここで ParGen は公開パラメータの生成，KGen は鍵ペア生成，Sign は署名生成，Verify は署名検証，CSign はコミットメントへの署名生成，CVerify はコミットメントに対する署名検証のアルゴリズムを指す．

本稿では Bobolz らの構成を，群構造維持署名ではなく，メッセージがスカラーな署名方式でも使えるように一般化した．構成の変更点は以下に示すように PresPolicy で Σ_I.Sign を利用していたのを Σ_I.CSign を用いるようにしたのみである．

$\mathsf{PresPolicy}(\{ipk_i\}) : \sigma_i \leftarrow \Sigma_V.\mathsf{CSign}(pp, vsk, ipk_i)$. Return $pol = (vpk, \{ipk_i, \sigma_i\})$.

3.1.3 安全性証明

前節で示した一般的構成が 3.1.1 節で述べた安全性の定義を満たすことを示す．Correctness については，プロトコル仕様に従う場合，ユーザが計算した提示トークンは検証者によって受け入れられることは明らかである．

次に Unforgeability について示す．

定理 11. もし Σ_I と Σ_V が EUF-CMA 安全であり，NIZK が Zero-Knowledge と Simulation-Sound Extractable であるならば，図 3 に示した一般的構成は Registered Key Model において Unforgeability を満たす.

一般的構成の Unforgeability を破る攻撃者は以下の 2 つのタイプに分類できる.

タイプ 1 challenge policy pol^* に含まれない公開鍵 ipk' に対する Σ_V 署名を偽造する攻撃者.

タイプ 2 ある honest な発行元の公開鍵 ipk_i に対して，Σ_I 署名の証明書を偽造する攻撃者.

以下，タイプ 1 攻撃者が存在するなら Σ_V の EUF-CMA 安全性が，破られることを示す．タイプ 2 攻撃者が存在する場合に Σ_I の EUF-CMA 安全性が破られることは [5] の証明と変わらないため，ここでは省略する.

証明. 攻撃者を確率的多項式時間計算能力の攻撃者とする．まず，$\mathcal{O}_{\mathsf{present}}$ によって生成される全ての NIZK の出力 pt をシミュレートすることで Unforgeability ゲームの改変を行う．NIZK はゼロ知識であるため，この変更による成功確率の増加は無視できる．以下では，改変されたゲームにおける A の成功確率が無視できるほど小さいことを示す.

改変された Unforgeability ゲームでは A は pol^* と $(pt^*, \phi^*, \mathsf{ctx}^*)$ を出力する．もし A が勝利するなら，我々は pt^* を NIZK の extractor に適用し，witness $(\sigma, ipk, cred, \mathbf{m})$ を抽出することができる.

A が勝利するが，extractor が有効な witness の抽出に失敗する事象を extractfail とする．A が勝利し，extractor による抽出に失敗せず，honest な発行元のものではない公開鍵 $(ipk = g_2^{isk} \notin \{ipk_i\})$ を抽出する事象を polforge とする．A が勝利し，extractor による抽出に失敗せず，honest な発行元の公開鍵の一つを抽出する事象を credforge とする.

これらの定義を用いると $\Pr[\mathsf{A\ wins}] \leq \Pr[\mathsf{A\ wins} \wedge \neg\mathsf{extractfail}] + \Pr[\mathsf{extractfail}] = \Pr[\mathsf{polforge}] + \Pr[\mathsf{credforge}] + \Pr[\mathsf{extractfail}]$ となる．NIZK は Simulation-Sound Extractable であるので，$\Pr[\mathsf{extractfail}]$ は無視できる．また，$\Pr[\mathsf{credforge}]$ については [5] により無視できることが証明されている.

よって $\Pr[\mathsf{polforge}]$ が無視できることを示せば証明が完了する.

タイプ 1 攻撃者. Σ_V の EUF-CMA 安全性に対して，攻撃者 B を以下のように構成する:

- B は，公開パラメータ pp および公開鍵 pk と，署名オラクル $\mathcal{O}_{\mathrm{sign}}$ へのアクセスを受け取る.
- B は, 鍵ペア $(isk_i, ipk_i) \xleftarrow{\$} \mathsf{IKGen}(pp), i \in [n_I]$ を生成する．その後, $\mathsf{A}^{\mathcal{O}_{\mathrm{issue}}, \mathcal{O}_{\mathrm{present}}, \mathcal{O}_{\mathrm{reveal}}}(pp, \{ipk_i\}_{i=1}^{n_I})$ を実行し，(I^*, st) を受け取る．なお A からのオラクルクエリに対しては，B は $\{isk_i\}$ を用いて正直に応答できる.
- I^* が honest でない発行元の公開鍵を含む場合，これは A の勝利条件に反するため，B は中断する.
- B は，I^* に含まれるすべての ipk_i に対応する isk_i を特定する．Honest な発行元の公開鍵は B 自身が生成しているのでこれは実行可能である.

- 特定したすべての isk_i を $\mathcal{O}_{\mathrm{sign}}$ へクエリし，対応する署名 σ_i を得る．そしてポリシー $pol^* = (pk, \{(ipk_i, \sigma_i)\})$ を作成する．ここでコミットメント付き署名 Σ_V の定義より，$\mathcal{O}_{\mathrm{sign}}$ から得られた $\sigma_i = \Sigma_V.\mathsf{Sign}(vsk, isk_i)$ は $\Sigma_V.\mathsf{CSign}(vsk, ipk_i)$ と一致するため，上のように作られた pol^* は正当なポリシーに等しい．
- B は，$\mathsf{A}^{\mathcal{O}_{\mathrm{issue}}, \mathcal{O}_{\mathrm{present}}, \mathcal{O}_{\mathrm{reveal}}}(st, pol^*)$ を実行して $(pt^*, \phi^*, \mathsf{ctx}^*)$ を得る．
- A の勝利条件が満たされない場合，B は中断する．
- それ以外の場合，B は extractor を用いて pt^* から $(\sigma^*, ipk^*, cred^*, \mathbf{m}^*)$ を抽出する．
- ここで Registered Key Model を前提としているため，[27] での議論と同様に，B は ipk^* の登録時に付与された知識証明から isk^* を抽出できる．
- $ipk^* \notin \{ipk_i\}_{i=1}^{n_I}$ であれば，B は (isk^*, σ^*) を偽造署名として出力する．

構成より，$\Pr[\text{B wins}] = \Pr[\mathsf{polforge}]$ が成り立つ（注意：polforge が生じる場合，B は isk^* に対する署名を $\mathcal{O}_{\mathrm{sign}}$ へ要求していない）．しかし，Σ_V が EUF-CMA 安全であることから，$\Pr[\mathsf{polforge}]$ は無視可能である． □

次に，Unlinkability について示す．定理 12 の証明は [5] と同じため省略する．

定理 12. もし NIZK が Zero-Knowledge であれば，図 3 に示した一般的構成は Unlinkability を満たす．

3.2 Bobolz らによる Issuer-Hiding AC

本節では，Bobolz らによる Groth 署名 [4] を用いた Issuer-Hiding AC の構成 [5] について説明する．

Bobolz らは，2.7 節で述べた Groth 署名を用いて Issuer-Hiding AC を構成した．その際，Groth_1 に加え，G_1 要素と G_2 要素を入れ替えた Groth 署名 Groth_2 も用いる．Bobolz らによる Issuer-Hiding AC の構成を示す．

- $\mathsf{ParGen}(1^\lambda)$: 公開パラメータを $pp = (G_1, G_2, G_{\mathrm{T}}, \mathbf{e}, p, g_1, g_2, y_1, y_2, (h_i)_{i=1}^{l})$ で生成する．なお，$y_1 \xleftarrow{\$} G_1, y_2 \xleftarrow{\$} G_2$ はランダムに選ばれた要素である．
- $\mathsf{IKGen}(pp)$: 発行元の鍵ペアを $(isk, ipk) \xleftarrow{\$} \mathsf{Groth}_1.\mathsf{KGen}(pp)$ で生成する．
- $\mathsf{Issue}(pp, isk, M)$: 発行元が証明書 $cred \xleftarrow{\$} \mathsf{Groth}_1.\mathsf{Sign}(pp, isk, \prod_{i=1}^{L} h_i^{m_i})$ を発行する．
- $\mathsf{VfCred}(pp, cred, M, ipk)$: 証明書 $cred$ を $\mathsf{Groth}_1.\mathsf{Verify}(pp, ipk, cred, \prod_{i=1}^{L} h_i^{m_i})$ で検証し，返り値を返す．
- $\mathsf{Prespolicy}(\{ipk_i\})$: 検証者の鍵ペアを $(vsk, vpk) \xleftarrow{\$} \mathsf{Groth}_2.\mathsf{KGen}(pp)$ で生成し，ポリシーに含める発行元の公開鍵 ipk_i に対して署名を $\sigma_i = (R_i, \tilde{S}_i, \tilde{T}_i) \xleftarrow{\$} \mathsf{Groth}_2.\mathsf{Sign}(pp, u, ipk_i)$ で生成する．公開鍵 ipk_i と署名 σ_i をまとめてポリシー $pol = (vpk, \{ipk_i, \sigma_i\})$ を作成する．
- $\mathsf{VfPolicy}(pol, \{ipk_i\})$: ポリシーに含まれる公開鍵と署名を $\mathsf{Groth}_2.\mathsf{Verify}(pp, vpk, \sigma_i, ipk_i)$ で

検証し，全ての発行元について 1 なら 1 を，そうでなければ 0 を返す．

- $\mathsf{Present}(pp, cred, ipk, M, \phi, pol, \mathsf{ctx})$: j を実際の発行元の添字とする．つまり，$ipk_j = ipk$ とする．また，$cred$ と σ_j を $(\tilde{R}, S, T) \xleftarrow{\$} \mathsf{Groth_1.Rand}(pp, cred)$ と $(R_j, \tilde{S}_j, \tilde{T}_j) \xleftarrow{\$} \mathsf{Groth_2.Rand}(pp, \sigma_j)$ でランダム化する．

  ランダムで秘匿する必要のある値 $\alpha, \beta, \gamma, \delta \xleftarrow{\$} \mathbb{Z}_p^*$ を選び，以下の計算をする．
  - 秘匿化された証明書 $(\tilde{R}, S', T') := (\tilde{R}, S^{1/\alpha}, T^{1/\beta})$ は，ipk_j のもとで $\prod_{i=1}^{L} h_i^{m_i}$ 上において定義され，発行元の秘匿化された公開鍵 $ipk'_j := ipk_j^{1/\gamma}$ を用いる．
  - 秘匿化された発行元の公開鍵に対する署名 $(R_j, \tilde{S}_j, \tilde{T}'_j) := (R_j, \tilde{S}_j, \tilde{T}_j^{1/\delta})$ は，検証者の公開鍵 vpk のもとで，発行元の公開鍵 ipk_j に基づいて定義される．

  証明 π を計算する．

$$
\begin{aligned}
&\pi \xleftarrow{\$} \mathrm{NIZK}[(\alpha, \beta, \gamma, \delta, \mathbf{m}) : \\
&\mathbf{e}(S', \tilde{R})^{\alpha} = \mathbf{e}(y_1, g_2) \cdot \mathbf{e}(g_1, ipk'_j)^{\gamma} \wedge \\
&\mathbf{e}(T', \tilde{R})^{\beta} = \mathbf{e}(y_1, ipk'_j)^{\gamma} \cdot \mathbf{e}(\prod_{i=1}^{\ell} h_i^{m_i}, g_2) \wedge \\
&\mathbf{e}(R_j, \tilde{S}_j) = \mathbf{e}(g_1, y_2) \cdot \mathbf{e}(vpk, g_2) \wedge \\
&\mathbf{e}(R_j, \tilde{T}'_j)^{\delta} = \mathbf{e}(vpk, y_2) \cdot \mathbf{e}(g_1, ipk'_j)^{\gamma} \wedge \\
&\text{Attribute check: } \phi(\mathbf{m}) = 1](pol, \phi, \mathsf{ctx})
\end{aligned}
\tag{1}
$$

  $pt = ((\tilde{R}, S', T'), ipk'_j, (R_j, \tilde{S}_j, \tilde{T}'_j), \pi)$ を返す．

- $\mathsf{Verify}(pt, list, \phi)$: もし pt が正しいと検証された場合 1 を，それ以外の場合は 0 を返す．

この構成では，具体的な NIZK の計算方法を示していない．また，属性を全て隠すようになっているため，選択的開示が効率的ではない．そこで，次小節では，効率的な選択的開示を行いつつ，NIZK の具体的な計算方法を示す．

3.2.1 Bobolz らの Issuer-Hiding AC の改良版

前小節で示した通り，Bobolz らの Issuer-Hiding AC の構成では，選択的開示の証明が非効率であり，かつ NIZK が具体的にどのように計算されるか示されていないので，Camenisch らの論文 [28] を参考に紹介する．

- $\mathsf{Present}(cred, ipk, \{m_i\}_{i \in [1,l]}, \phi, pol, \mathsf{ctx})$: 証明 π の計算前までは変更がない．

  J を開示するメッセージの添字の集合，I を開示しないメッセージの添え字の集合とし，公開しないランダムな値 $r_1, r_2, r_3, r_4, r_{5,i} \xleftarrow{\$} \mathbb{Z}_p^*, \quad i \in I$ を生成する．

$$
\begin{aligned}
U_1 &= \mathbf{e}(S', \tilde{R})^{r_1} \cdot \mathbf{e}(g_1, ipk'_j)^{-r_3} \\
U_2 &= \mathbf{e}(T', \tilde{R})^{r_2} \cdot \mathbf{e}(y_1, ipk'_j)^{-r_3} \cdot \mathbf{e}(\prod_{i \in I} h_i^{-r_{5,i}}, g_2) \\
U_3 &= \mathbf{e}(R_j, \tilde{T}'_j)^{r_4} \cdot \mathbf{e}(g_1, ipk'_j)^{-r_3}
\end{aligned}
$$

をそれぞれ計算する．

$$c := H((\tilde{R}, S', T'), ipk'_j, (R_j, \tilde{S}_j, \tilde{T}'_j), \{h_i\}_{i\in l}, \{m_i\}_{i\in J}, U_1, U_2, U_3, pol)$$

を計算し，

$$\begin{aligned} s_1 &= r_1 + c \cdot \alpha \\ s_2 &= r_2 + c \cdot \beta \\ s_3 &= r_3 + c \cdot \gamma \\ s_4 &= r_4 + c \cdot \delta \\ s_{5,i} &= r_{5,i} + c \cdot m_i \quad (i \in I) \end{aligned}$$

とする．$\pi = (s_1, s_2, s_3, s_4, s_{5,i}, c)$ とし，$pt = ((\tilde{R}', S'', T''), ipk'_j, (R'_j, \tilde{S}'_j, \tilde{T}''_j), \pi)$ を返す．

- $\mathsf{Verify}(pt, pol, \phi) : \pi$ の検証として，

$$\begin{aligned} U'_1 &= \mathbf{e}(S', \tilde{R})^{s_1} \cdot \mathbf{e}(g_1, ipk'_j)^{-s_3} \cdot \mathbf{e}(y_1, g_2)^{-c} \\ U'_2 &= \mathbf{e}(T', \tilde{R})^{s_2} \cdot \mathbf{e}(y_1, ipk'_j)^{-s_3} \cdot \mathbf{e}(\prod_{i\in I} h_i^{-s_{5,i}}, g_2) \cdot \mathbf{e}(\prod_{i\in J} h_i^{m_i}, g_2)^{-c} \\ U'_3 &= \mathbf{e}(R_j, \tilde{T}'_j)^{s_4} \cdot \mathbf{e}(g_1, ipk'_j)^{-s_3} \cdot \mathbf{e}(vpk, y_2)^{-c} \end{aligned}$$

を計算し，$c' := H((\tilde{R}, S', T'), ipk'_j, (R_j, \tilde{S}_j, \tilde{T}'_j), \{h_i\}_{i\in l}, \{m_i\}_{i\in J}, U'_1, U'_2, U'_3, pol)$ を求め，$c = c'$ であることを確認する．また，

$$e(R_j, \tilde{S}_j) = e(G, y_2) \cdot e(vpk, g_2) \tag{2}$$

が成り立つことを確認する．
全て成り立てば 1 をそうでなければ 0 を返す．

3.3 Sanders らによる Issuer-Hiding AC

本節では，Sanders らによる PS 署名を用いた Issuer-Hiding AC の構成 [7] について説明する．

Sanders らは，Issuer-Hiding AC を構成するために PS 署名では KeyGen で作成していた X を ParGen で公開パラメータとして生成するように変更している．以下に Sanders らの Issuer-Hiding AC の構成を示す．ただし，ポリシーに含まれる発行元の公開鍵の集合を $\mathcal{P}$ とする．

- $\mathsf{PS.ParGen}(1^\lambda)$：位数 p の双線形群を構成する公開パラメータ $pp = (G_1, G_2, G_T, \mathbf{e}, p, g_1, g_2, X_1, X_2)$ を生成する．ここで，$g_1 \in G_1, g_2 \in G_2$ はそれぞれの生成元である．また，X_1, X_2 はランダムな値 $x \xleftarrow{\$} \mathbb{Z}_p$ を用いて，$X_1 = g_1^x, X_2 = g_2^x$ と計算する．公開パラメータ pp を返す．
- $\mathsf{PS.IKGen}(pp)$：ランダムに $y_i \xleftarrow{\$} \mathbb{Z}_p^l$ を選び，秘密鍵 $isk = (\{y_i\}_{i\in[l]})$，公開鍵 $ipk = (\{Y_i = g_2^{y_i}\}_{i\in[l]})$ を生成する．ここで，l は属性数である．(isk, ipk) を返す．

- PS.Issue$(pp, isk, \mathbf{m})$：属性 $\mathbf{m} = (m_1, m_2, \ldots, m_l)$ とランダムな値 $r \xleftarrow{\$} \mathbb{Z}_p$ に対し，以下の計算を行う．

$$\sigma_1 = g_1^r$$
$$\sigma_2 = X_1^r \cdot g_1^{r \cdot \sum_{i=1}^{l} y_i \cdot m_i}$$

を計算する．署名 $\sigma = (\sigma_1, \sigma_2)$ を返す．

- PS.VfCred$(pp, ipk, \sigma, \mathbf{m})$：以下のペアリングの等式が成り立つか確認する．

$$\mathbf{e}(\sigma_1, X_2 \cdot \prod_{i=1}^{l} Y_i^{m_i}) \stackrel{?}{=} \mathbf{e}(\sigma_2, g_2)$$

- PS.PresPolicy$(\mathcal{P})$：$l+1$ 個のランダムな値 $a, b_i \xleftarrow{\$} \mathbb{Z}_p$ を選び，$S = g_2^a, B_i = g_2^{b_i(n_I-1)}, T_{j,i} = (Y_{j,i} \cdot g_2^{b_i})^a$ を全ての $i \in [l], j \in [n_I]$ について計算する．ポリシーの秘密部分を $sk_{\mathcal{P}} = (a, b_i)$，公開部分を $pk_{\mathcal{P}} = (S, \{B_i\}_{i \in [l]}, \{T_{j,i}\}_{i \in [l], j \in [n_I]})$ とする．$T_{j,i}$ が正しく計算されていることを示すゼロ知識証明 π を以下のように生成する．
  - $l+1$ 個のランダムな値 $r_0, \ldots, r_l \xleftarrow{\$} \mathbb{Z}_p^{l+1}$ を選び，$K = S^{r_0}, K_{b_i} = g_2^{r_i(n_I-1)}, K_{j,i} = T_{j,i}^{r_0} \cdot g_2^{r_i}$ を全ての $i \in [l], j \in [n_I]$ について計算する．
  - チャレンジ $c = H(pk_{\mathcal{P}}, S, K, \{K_{b_i}\}_{i \in [l]}, \{K_{j,1}\}_{j \in [n_I]}, \ldots, \{K_{j,l}\}_{j \in [n_I]})$ を計算する．
  - $z_0 = r_0 + a^{-1} \cdot c, z_i = r_i + b_i \cdot c$ を全ての $i \in [l]$ について計算する．
  - 証明 $\pi = (c, z_0, \{z_i\}_{i \in [l]})$ とする．

  $(sk_{\mathcal{P}}, pk_{\mathcal{P}}, \pi)$ を返す．

- PS.VfPolicy$(\mathcal{P}, pk_{\mathcal{P}}, \pi)$：以下の等式が成り立つか確認する．

$$c \stackrel{?}{=} H(pk_{\mathcal{P}}, S^{z_0} \cdot g_2^{-c}, \{g_2^{z_i(n_I-1)} \cdot B_i^c\}_{i \in [l]}, \{T_{j,i}^{z_0} \cdot g_2^{z_1} \cdot Y_{j,1}^{-c}\}_{j \in [n_I]}, \ldots, \{T_{j,l}^{z_0} \cdot g_2^{z_l} \cdot Y_{j,l}^{-c}\}_{j \in [n_I]})$$

- PS.Present$(\mathbf{m}, (\sigma_1, \sigma_2), ipk, \mathcal{P}, pk_{\mathcal{P}})$：署名 (σ_1, σ_2) と関連づいている公開鍵 ipk がポリシー $\mathcal{P}$ と互換性があるか確認する．つまり，$\exists j_k \in [n_I], \forall i \in [l]$ について $Y_i = Y_{j_k,i}$ となっているか確認する．ランダムな値 $r, t \in \mathbb{Z}_p^{*2}$ を選び $\sigma_1' \leftarrow \sigma_1^r, \sigma_2' \leftarrow \sigma_2^r \cdot (\sigma_1')^{-t}$ を計算する．そして，

$$\tilde{\sigma} \leftarrow S^t \prod_{i=1}^{n} \left[\prod_{j \in [n_I] \setminus j_k} T_{j,i} \right]^{m_i}$$

を計算する．$\{m_i\}_{i \in I}$ についての証明

$$\pi' \leftarrow \mathrm{PoK}[\prod_{i \in I} \mathbf{e}(\sigma_1', B_i \prod_{j \in [n_I]} Y_{j,i})^{m_i}$$
$$= \mathbf{e}(\sigma_2', g_2) \cdot \mathbf{e}(\sigma_1', X_2^{-1} \tilde{\sigma}^{\frac{1}{a}} \prod_{i \in J} [B_i \prod_{j \in [n_I]} Y_{j,i}]^{-m_i})]$$

を生成する．具体的には，以下の手順で証明を生成する．

– d' 個のランダムな値 $\{k_i\}_{i\in I}$ を生成し，$K = \prod_{i\in I} \mathbf{e}(\sigma'_1, B_i \prod_{j\in[n_I]} Y_{j,i})^{k_i}$ を計算する．
– チャレンジ $c = H(K, pk_{\mathcal{P}}, (\sigma'_1, \sigma'_2, \tilde{\sigma}))$ を計算する．
– $z_i = k_i + m_i \cdot c$ を全ての $i \in I$ について計算する．
– $\pi = (c, \{z_i\}_{i\in I})$ とする．

$pt = ((\sigma'_1, \sigma'_2, \tilde{\sigma}), \pi, \{m_i\}_{i\in I})$ を返す．

- $\mathsf{Verify}(pt, \mathcal{P}, sk_{\mathcal{P}})$：$\sigma'_1 \neq 1$ であることを確認する．次に，

$$T = \mathbf{e}(\sigma'_2, g_2) \cdot \mathbf{e}(\sigma'_1, X_2^{-1}\tilde{\sigma}^{\frac{1}{a}} \prod_{i\in J}[g_2^{b_i(n_I-1)} \prod_{j\in[n_I]} Y_{j,i}]^{-m_i})$$

を計算する．最後に，$K' = T^{-c} \prod_{i\in I} \mathbf{e}(\sigma'_1, g_2^{b_i(n_I-1)} \prod_{j\in[n_I]} Y_{j,i})^{z_i}$ を計算する．$c \stackrel{?}{=} H(K', pk_{\mathcal{P}}, (\sigma'_1, \sigma'_2, \tilde{\sigma}))$ が成り立てば 1 を，そうでなければ 0 を返す．

3.4 Katz らによる Issuer-Hiding AC

本節では，Katz らによる BBS 署名を用いた Issuer-Hiding AC の構成 [8] について説明する．

Katz らの構成は，Sanders らの構成 [7] をもとにしており，Sanders らの構成と同様に，BBS 署名を用いない検証者による発行元の公開鍵に対する署名を利用して Issuer-Hiding AC を実現している．以下に Katz らの Issuer-Hiding AC の構成を示す．

- $\mathsf{Katz.ParGen}(1^\lambda)$：位数 p の双線形群を構成する公開パラメータ $pp = (G_1, G_2, G_T, \mathbf{e}, p, g_1, g_2, \{h_i\}_{i=1}^{l})$ を生成する．公開パラメータ pp を返す．
- $\mathsf{Katz.IKGen}(pp)$：$(isk, ipk) \stackrel{\$}{\leftarrow} \mathsf{BBS.KGen}(pp)$ とする．(isk, ipk) を返す．
- $\mathsf{Katz.Issue}(pp, isk, \mathbf{m})$：属性 $\mathbf{m} = (m_1, m_2, \ldots, m_l)$ に対し，$cred \stackrel{\$}{\leftarrow} \mathsf{BBS.Sign}(pp, isk, \mathbf{m})$ を計算する．署名 $cred$ を返す．
- $\mathsf{Katz.VfCred}(pp, ipk, cred, \mathbf{m})$：$\mathsf{BBS.Verify}(pp, ipk, cred, \mathbf{m}) = 1$ が成り立てば 1 を，そうでなければ 0 を返す．
- $\mathsf{Katz.PresPolicy}(\{ipk_i\})$: ランダムな値 $a \stackrel{\$}{\leftarrow} \mathbb{Z}_p^*, b \stackrel{\$}{\leftarrow} \mathbb{Z}_p$ を生成し，$S = g_2^a, T_i = (ipk_i \cdot g_2^b)^a$ を全ての $i \in [n_I]$ について計算する．ランダムな値 $\alpha, \beta \stackrel{\$}{\leftarrow} \mathbb{Z}_p$ を生成し，$U_1 = g_2^\alpha, U_{2,i} = T_i^\alpha g_2^\beta$ を全ての $i \in [n_I]$ について計算し，$c = H(\{ipk_i\}_{i\in[n_I]}, S, \{T_i\}_{i\in[n_I]}, U_1, \{U_{2,i}\}_{i\in[n_I]})$ を計算する．$s = \alpha + c/a, t = \beta - b \cdot c$ とし，証明 $\pi = (c, s, t)$ とする．ポリシーの秘密部分を $sk_{\mathcal{P}} = (a, b)$，公開部分を $pk_{\mathcal{P}} = (S, \{T_i\}_{i\in[n_I]}, \pi)$ とする．$(sk_{\mathcal{P}}, pk_{\mathcal{P}})$ を返す．
- $\mathsf{Katz.VfPolicy}((\mathcal{P}, pk_{\mathcal{P}}))$: $n_I = |\mathcal{P}|, S \neq 1$ であることと，$c \stackrel{?}{=} H(\{pk_i\}_{i\in[n_I]}, S, \{T_i\}_{i\in[n_I]}, S^s \cdot g_2^{-c}, \{T_i^s \cdot g_2^t \cdot ipk_i^{-c}\}_{i\in[n_I]})$ が成り立つか確認する．
- $\mathsf{Katz.Present}(\mathcal{P}, J, \mathbf{m}, \sigma, ipk)$: 実際の発行元の公開鍵を ipk_j とする．ランダムな値 $r_1, r_2 \stackrel{\$}{\leftarrow} \mathbb{Z}_p^*, t \stackrel{\$}{\leftarrow} \mathbb{Z}_p$ を生成する．$\bar{A} = A^{r_1/r_2}, D = C(\mathbf{m})^{1/r_2}, \bar{B} = D^{r_1}\bar{A}^{-e-t}$ とし，$\tilde{\sigma} = S^t \cdot \prod_{i\neq j} T_i$ とする．

  ランダムな値 $\alpha, \beta, \gamma, \delta_i \stackrel{\$}{\leftarrow} \mathbb{Z}_p^*(\forall i \in I)$ を生成し，$U_1 := D^\alpha \bar{A}^\beta, U_2 := D^\gamma \prod_{i\in I} h_i^{\delta_i}$ を計算し，$c := H(\{h_i\}_{i\in[l]}, \{m_i\}_{i\in J}, \bar{A}, \bar{B}, D, U_1, U_2)$ を計算する．

$s := \alpha + r_1 \cdot c, t := \beta - e \cdot c, z := \gamma + r_2 \cdot c, u_i := \delta_i - m_i \cdot c (\forall i \in I)$ を計算し，証明 $\pi = (c, s, t, z, \{u_i\}_{i \in I})$ とする．$pt = (D, \bar{A}, \bar{B}, \tilde{\sigma}, \pi)$ を返す．

- $\mathsf{Verify}(sk_{\mathcal{P}}, pk_{\mathcal{P}}, \{m_i\}_{i \in J}, pt)$: $\bar{A} \neq 1$ と，$\mathbf{e}(\bar{A}, \tilde{\sigma}^{-1/a} \cdot g_2^{(n_I - 1) \cdot b} \cdot \prod_{i \in [n_I]} ipk_i) \stackrel{?}{=} \mathbf{e}(\bar{B}, g_2)$ が成り立ち，$c \stackrel{?}{=} H(\{h_i\}_{i \in [l]}, \{m_i\}_{i \in J}, \bar{A}, \bar{B}, D, D^s \bar{A}^t \bar{B}^{-c}, D^z \prod_{i \in I} h_i^{u_i} C_J(\mathbf{m})^{-c})$ が成り立つか確認する．

4 BBS 署名を用いた Issuer-Hiding AC の提案

本章では，3.1 節で示した Issuer-Hiding AC の一般的構成を BBS 署名とシグマプロトコルで具体化する．また，この構成が Issuer-Hiding AC の安全性を満たすことを示す．

具体的には，BBS 署名を利用した Issuer-Hiding AC を構成するにあたり，Bobolz らが示した Groth 署名を用いた具体的な Issuer-Hiding AC の構成 [5, Construction 2] のように，検証者が G_1 と G_2 の要素を入れ替えた形で署名を生成することで構成する．これにより，検証者が信頼できる発行元の公開鍵に対し署名を行い，ユーザは実際の発行元の公開鍵をランダム化した上で検証者の署名が行われていることを証明することで発行元を秘匿することができる．この方式の利点として，既存の BBS 署名の構成を流用できることが挙げられる．

4.1 Issuer-Hiding AC の構成

検証者の鍵生成および，検証者による信頼できる発行元の公開鍵への署名とポリシー作成のために以下のようなアルゴリズムを定義する．

- $\mathsf{VKGen}(pp)$. $(vsk, vpk_1) \xleftarrow{\$} \mathsf{BBS}_V.\mathsf{KGen}(pp)$ を計算する．また，$vpk_2 = \bar{g}_2^{vsk}$ とする．鍵ペア $(vsk, \mathbf{vpks} = (vpk_1, vpk_2))$ を返す．
- $\mathsf{PresPolicy}(\{ipk_i\})$: $\sigma_i \leftarrow \mathsf{BBS}_V.\mathsf{Sign}(vsk, ipk_i), (i \in [n_I])$ を求める．$pol = (\mathbf{vpk}, \{ipk_i, \sigma_i\}_{i \in [n_I]})$ を返す．
- $\mathsf{VfPolicy}(pol, \{ipk_i\})$: $\mathbf{e}(vpk_1, \bar{g}_2) = \mathbf{e}(\bar{g}_1, vpk_2)$ をチェックする．$\mathbf{e}(vpk_1 \cdot \bar{g}_1^{e_i}, A_i) = \mathbf{e}(\bar{g}_1, \bar{g}_2 \cdot ipk_i)$ が全ての $i \in [n_I]$ で成り立てば 1，そうでなければ 0 を返す．

ここで示した構成の署名検証をそのまま検証者に行ってもらおうとすると隠したい値が隠せていない．ゆえに情報を隠すことを考える必要がある．実際の発行元の公開鍵を ipk_j とする．

証明したい式を Camenisch-Stadler 記法 [29] で表すと以下のようになる．

$$
\begin{aligned}
\pi \leftarrow & \mathrm{SPK}[(A, e, ipk_j, A_j, e_j, \{m_k\}_{k \in I}) : \\
& \mathbf{e}(A, ipk_j \cdot g_2^e) = \mathbf{e}\left(g_1 \cdot \prod_{i=1}^{l} h_i^{m_i}, g_2\right) \wedge \\
& \mathbf{e}(vpk \cdot \bar{g_1}^{e_j}, A_j) = \mathbf{e}(\bar{g_1}, \bar{g_2} \cdot ipk_j) \wedge \\
& \phi(\mathbf{m}) = 1](pol, \phi, \mathsf{ctx})
\end{aligned}
\tag{3}
$$

ここで，I は非開示にする属性の添え字の集合とし，$\phi(\mathbf{m}) = 1$ は，各属性の値が検証者が求める属性値を満たすことを示す．また，ctx はセッション ID やランダムなナンスなどのリプレイ攻撃や似た攻撃を防ぐためのコンテキスト情報である．

式 (3) にシグマプロトコルを適用することで，効率的な証明が可能となる．BBS 署名を用いた Issuer-Hiding AC の構成を示す．なお，IKGen, Issue, VfCred は一般的構成の Σ を BBS に置き換えただけであり，VKGen, PresPolicy, VfPolicy はすでに示した通りであるため省略する．ま

た，J は開示する属性の添え字の集合とし，$C_J(\mathbf{m}) = g_1 \prod_{j \in J} h_j^{m_j}$ とする．

- $\mathsf{ParGen}(1^\lambda)$. $pp \leftarrow (G_1, G_2, G_T, p, \mathbf{e}, g_1 \in G_1, \bar{g}_1 \in G_1, g_2 \in G_2, \bar{g}_2 \in G_2, \{h_i\}_{i=1}^l \in G_1)$.
- $\mathsf{IKGen}(pp)$. $(isk, ipk) \xleftarrow{\$} \mathsf{BBS}_I.\mathsf{KGen}(pp)$ とする．鍵ペア (isk, ipk) を返す．
- $\mathsf{Issue}(isk, \mathbf{m})$. 証明書 $cred \xleftarrow{\$} \mathsf{BBS}_I.\mathsf{Sign}(pp, isk, \mathbf{m})$ を返す．
- $\mathsf{VfCred}(cred, \mathbf{m}, ipk)$. $\mathsf{BBS}_I.\mathsf{Verify}(pp, ipk, cred, \mathbf{m}) = 1$ が成り立てば 1 を，そうでなければ 0 を返す．
- $\mathsf{VKGen}(pp)$. $(vsk, vpk_1) \xleftarrow{\$} \mathsf{BBS}_V.\mathsf{KGen}(pp)$ を計算する．また，$vpk_2 = \bar{g}_2^{vsk}$ とする．鍵ペア $(vsk, \mathbf{vpks} = (vsk_1, vsk_2))$ と証明 π_{vsk} を返す．
- $\mathsf{PresPolicy}(\{ipk_i\})$: $\sigma_i \leftarrow \mathsf{BBS}_V.\mathsf{Sign}(vsk, ipk_i), (i \in [n_I])$ を求める．$pol = (\mathbf{vpk}, \{ipk_i, \sigma_i\}_{i \in [n_I]})$ を返す．
- $\mathsf{VfPolicy}(pol, \{ipk_i\})$: π_{vsk} をチェックする．もし $\mathbf{e}(vpk_1 \cdot \bar{g}_1^{e_i}, A_i) = \mathbf{e}(\bar{g}_1, \bar{g}_2 \cdot ipk_i)$ が全ての $i \in [n_I]$ で成り立てば 1 を，そうでなければ 0 を返す．
- $\mathsf{Present}(pp, cred, ipk, \mathbf{m}, \phi, pol, \mathsf{ctx})$. $(\mathbf{vpk} = (vpk_1, vpk_2), \{ipk_i, \sigma_i\}) \leftarrow pol$ とする．$ipk_i = ipk$ である添え字 i を j とする．
  ランダムな $r, r_1, r_2, r_3 \xleftarrow{\$} \mathbb{Z}_p^*$ を導入し，$ipk'_j = ipk_j^r, \bar{A} = A^{r_1/(r \cdot r_2)}, D = (g_1 \cdot \prod_{i=1}^l h_i^{m_i})^{1/r_2}, \bar{B} = D^{r_1} \cdot \bar{A}^{-e \cdot r}$ とする．また，$\bar{A}_j = A_j^{r/r_3}, D_j = (\bar{g}_2 \cdot ipk_j)^{1/r_3}, \bar{B}_j = D_j^r \cdot \bar{A}_j^{-e_j}$ とする．
  $U_1 = D^\alpha \bar{A}^\beta, U_2 = D^\gamma \prod_{i \in I} h_i^{\delta_i}$ を計算する．ここで，$\alpha, \beta, \gamma, \delta_i \xleftarrow{\$} \mathbb{Z}_p^* \quad (\forall i \in I)$ である．また，$U_3 = D_j^{\alpha'} \cdot \bar{A}_j^{\beta'}, U_4 = D_j^{\gamma'} \cdot \bar{g}_2^{-\alpha'}$ を計算する．ここで，$\alpha', \beta', \gamma' \xleftarrow{\$} \mathbb{Z}_p^*$ である．
  チャレンジとしてハッシュ値

  $$c = H(\{h_i\}_{i \in [l]}, \{m_i\}_{i \in J}, \bar{A}, \bar{B}, D, U_1, U_2, ipk'_j, \bar{A}_j, \bar{B}_j, D_j, U_3, U_4, pol, \phi, \mathsf{ctx})$$

  を計算する．そして $s := \alpha + c \cdot r_1, t := \beta + c \cdot (-e \cdot r), z := \gamma + c \cdot (r_2), u_i := \delta_i + c \cdot (-m_i)$ を計算する．また，$s' := \alpha' + c \cdot r, t' := \beta' + c \cdot (-e_j), z' := \gamma' + c \cdot (r \cdot r_3)$ を計算する．$\pi = (s, t, z, (u_i)_{i \in I}, s', t', z', c)$ とし，$pt = (\mathbf{m}', \bar{A}, \bar{B}, D, ipk'_j, \bar{A}_j, \bar{B}_j, D_j, \pi)$ を提示トークンとして返す．
- $\mathsf{Verify}(pt, pol, \phi, \mathsf{ctx})$. 以下の式が正しく検証できれば 1 を返す．そうでなければ 0 を返す．

  $$\mathbf{e}(\bar{A}, ipk'_j) \stackrel{?}{=} \mathbf{e}(\bar{B}, g_2), \quad \mathbf{e}(vpk_1, \bar{A}_j) \stackrel{?}{=} \mathbf{e}(\bar{g}_1, \bar{B}_j)$$
  $$c \stackrel{?}{=} H(\{h_i\}_{i \in [l]}, \{m_i\}_{i \in J}, \bar{A}, \bar{B}, D, D^s \bar{A}^t \bar{B}^{-c}, D^z \prod_{i \in I} h_i^{u_i} \cdot C_J(\mathbf{m})^{-c},$$
  $$ipk'_j, \bar{A}_j, \bar{B}_j, D_j, D_j^{s'} \bar{A}_j^{t'} \bar{B}_j^{-c}, D_j^{z'} \bar{g}_2^{-s'} ipk_j'^{-c}, pol, \phi, \mathsf{ctx})$$

4.2　BBS 署名を用いた Issuer-Hiding AC の構成の安全性の証明

次に，4.1 節に示す構成が Issuer-Hiding AC の安全性を満たすことを示す．定理 11 と定理 12 より，BBS_I と BBS_V が EUF-CMA 安全であり，NIZK が Zero-Knowledge かつ Simulation-Sound Extractable であるならば，4.1 節に示した構成は Unforgeability と Unlinkability を満たすと言える．

2.5 節で示した通り，[2] により BBS_I が EUF-CMA 安全であることが示されている．また，BBS_V についても，EUF-CMA 安全であると言える．

次に，[30, Appendix B] を参考に Special Soundness を満たしていることを証明する．

等価でない 2 つのチャレンジ $c_1 \neq c_2$ に対して $\bar{A}, \bar{B}, D, U_1,$
$U_2, \bar{A}_j, \bar{B}_j, D_j, U_3, U_4$ が有効であることを示す，$s_1, s_2, t_1, t_2,$
$z_1, z_2, (u_{i,1})_{i\in I}, (u_{i,2})_{i\in I}, s'_1, s'_2, t'_1, t'_2, z'_1, z'_2$ および $\{m_i\}_{i\in J}$ とランダム化された発行元の公開鍵 ipk'_j が与えられたとする．すると，

$$
\begin{aligned}
\mathbf{e}(\bar{A}, ipk'_j) &= \mathbf{e}(\bar{B}, g_2) \\
D^{s_1} \cdot \bar{A}^{t_1} \cdot \bar{B}^{-c_1} &= U_1 = D^{s_2} \cdot \bar{A}^{t_2} \cdot \bar{B}^{-c_2} \\
D^{z_1} \prod_{i\in I} h_i^{u_{i,1}} \cdot C_J(\mathbf{m})^{-c_1} &= U_2 = D^{z_2} \prod_{i\in I} h_i^{u_{i,2}} \cdot C_J(\mathbf{m})^{-c_2} \\
\mathbf{e}(vpk_1, \bar{A}_j) &= \mathbf{e}(\bar{g}_1, \bar{B}_j) \\
D_j^{s'_1} \cdot \bar{A}_j^{t'_1} \cdot \bar{B}_j^{-c_1} &= U_3 = D_j^{s'_2} \cdot \bar{A}_j^{t'_2} \cdot \bar{B}_j^{-c_2} \\
D_j^{z'_1} \cdot \bar{g}_2^{-s'_1} \cdot ipk_j'^{-c_1} &= U_4 = D_j^{z'_2} \cdot \bar{g}_2^{-s'_2} \cdot ipk_j'^{-c_2}
\end{aligned}
$$

が成り立つ．ここで，$r_1 = (s_1 - s_2)/(c_1 - c_2)$，　$r_2 = (z_1 - z_2)/(c_1 - c_2)$，　$e' = (t_2 - t_1)/(c_1 - c_2)$，　$\mu_i = (u_{i,1} - u_{i,2})/(c_1 - c_2)$，　$r = (s'_1 - s'_2)/(c_1 - c_2)$，　$e_j = (t'_2 - t'_1)/(c_1 - c_2)$，　$r_3 = (z'_2 - z'_1)/(r(c_1 - c_2))$ なので，

$$
\begin{aligned}
\bar{B} &= D^{r_1} \cdot \bar{A}^{-e'}, \\
C_J(\mathbf{m}) &= D^{r_2} \cdot \prod_{i\in I} h_i^{-\mu_i}, \\
\bar{B}_j &= D^{r} \cdot \bar{A}_j^{-e_j}, \\
ipk'_j &= D_j^{r\cdot r_3} \cdot \bar{g}_2^{-r}
\end{aligned}
$$

なる $r_1, r_2, e', (\mu_i)_{i\in I}, r, e_j, r_3$ を抽出可能であり，$e = e'/r, ipk_j = ipk_j'^{1/r}$ なので，これらも抽出可能である．

もし $r_1 \neq 0, r_2 \neq 0$ であれば，$(A = \bar{A}^{r\cdot r_2/r_1}, e)$ は属性 $\mathbf{m}$ に対する有効な署名であり，$m_i = \mu_i$ が成り立つ．もし，$r_1 = 0$ であれば，$\mathbf{e}(\bar{A}, ipk'_j) = \mathbf{e}(\bar{A}^{-e'}, g_2)$ となり，$isk_j = -e$ が抽出できる．これは属性 $\{m_i\}_{i\in J}$ に対する有効な署名を生成することに等しい．もし，$r_2 = 0$

であれば，$g_1 \prod_{j \in J} h_j^{m_j} \cdot \prod_{i \in I} h_i^{-\mu_i} = D^{r_2} = 1$ となり，$g_1, (h_i)_{i=1}^l$ 間の非自明な線形関係を与える．

次に，もし $r \neq 0, r_3 \neq 0$ であれば，$(A_j = \bar{A}_j^{r_3/r}, e_j)$ は，属性 $ipk_j = ipk_j'^{1/r}$ に対する有効な署名である．もし $r = 0$ であれば，$\mathbf{e}(vpk_1, \bar{A}_j) = \mathbf{e}(\bar{g_1}, \bar{A}_j^{-e_j})$ となり，$vsk = -e_j$ が抽出できる．これは属性 ipk_j に対する有効な署名を生成することに等しい．もし，$r_3 = 0$ であれば，$\bar{g}_2 \cdot ipk_j'^{1/r} = D_j^{r_3} = 1$ となり，g_2, ipk_j 間の非自明な線形関係を与える．

よって，Special Soundness が満たされる．

次に，[28], [30] を参考に Honest-Verifier Zero-Knowledge (HVZK) を満たすことを示す．

ランダムな値 $x \xleftarrow{\$} \mathbb{Z}_p^*$ を用意し，$ipk = g_2^x$ とする．ランダムな値 $r, r_1 \xleftarrow{\$} \mathbb{Z}_p^*$ を用意し，$\bar{A} = g_1^{r_1/r}, \bar{B} = g_1^{r_1 \cdot x}, ipk' = ipk^r$ としてシミュレートする．次に，検証者の公開鍵 $\mathbf{vpk}$ から vpk_2 を取り出す．ランダムな値 $r_2, r_3 \xleftarrow{\$} \mathbb{Z}_p^*$ を用意し，$\bar{A}_j = \bar{g}_2^{r_2 \cdot r}, \bar{B}_j = vpk_2^{r_2 \cdot r}$ としてシミュレートする．最後にランダムな値 $D \xleftarrow{\$} G_1$ と $D' \xleftarrow{\$} G_2$ を用意し，ランダムなチャレンジ $c \xleftarrow{\$} \mathbb{Z}_p$ とランダムな値 $s, s', t, t', z, z', (u_i)_{i \in I} \xleftarrow{\$} \mathbb{Z}_p$ を生成する．そして

$$
\begin{aligned}
U_1 &= D^s \bar{A}^t \bar{B}^{-c}, \\
U_2 &= D^z \cdot \prod_{i \in I} h_i^{u_i} \cdot C_J(\mathbf{m})^{-c}, \\
U_3 &= D'^{s'} \bar{A}_j^{t'} \bar{B}_j^{-c}, \\
U_4 &= D'^{z'} \cdot \bar{g}_2^{-s'} \cdot ipk'^{-c}
\end{aligned}
$$

とする．すると，全ての検証式が成り立ち，全ての値が正しい分布と同じになっているため，シミュレートできている．

よって，HVZK が満たされる．

最後に，[22] を元に Simulation-Sound Extractability を満たすことを示す．[22, Definition 2] より，4.1 節に示した構成で用いられている NIZK は強 Fiat-Shamir 変換を利用していると言える．なぜなら，$\bar{A}, \bar{B}, D, \bar{A}_j, \bar{B}_j, D_j, ipk_j'$ は [22, Definition 2] における Y に対応しており，U_1, U_2, U_3, U_4 は A に対応しており，これらをハッシュ関数に入力してチャレンジ c を作成すれば良いからである．また，すでに Special Soundness と HVZK を満たしていることを示しているため，4.1 節に示した構成で用いられている NIZK は Simulation-Sound Extractable である．

以上より，定理 11 と定理 12 を満たすといえ，安全性が示された．

5 比較・評価

本章では，提案手法の評価を行う．具体的には，まず，Bobolz らの Groth 署名を用いた Issuer-Hiding AC の構成 [5]，Sanders らの PS 署名を用いた Issuer-Hiding AC の構成 [7]，Katz らの BBS 署名を用いた Issuer-Hiding AC の構成 [8] と提案手法を立脚する計算モデルの観点から比較する．また，それぞれの理論的な計算コストとデータサイズを比較する．さらに，それぞれを実装し，処理時間の計測を行う．

5.1 実行環境と設定

以下のスペックのマシンで発行元による証明書の作成（Sign），証明書の検証（VfCred），ポリシーの発行（PresPolicy），ポリシーの検証（VfPolicy），証明生成（Present），証明検証（Verify）の処理時間を計測した．

Mac Studio 2023, Apple M2 Max, 64GB, macOS Sqruoia 15.6.1, 1TB Strorage

全ての実装[*1]を Rust 言語で行い，暗号プリミティブは ark_bls12_381, ark_ff, ark_ec, ark_std, ark_serialize を用いた．ベンチマークの測定は，‘cargo bench’ コマンドを用い，各プロセスにおいて 100 回繰り返した実行時間の平均を計測した．証明書の作成と検証は属性数 l を，ポリシーの発行と検証はポリシーに含まれる発行元の数 n_I を，証明生成と検証は属性数 l，非開示にする属性数 d'，および n_I を変化させて計測した．全ての計測において，l は $5, 10, 15, 20, 25, 30, 35, 40, 45, 50$，$n_I$ は $5, 10, 50, 100, 500, 1000$，$d'$ は $3, 0.4l, l-3$ と変化させた．

5.2 評価結果

5.2.1 安全性証明モデル

Sanders らの構成，Katz らの構成，Bobolz らの構成は Generic Group Model (GGM) において安全性証明がつけられている．これに対し，我々の安全性証明は標準モデルに Registered Key Model を追加した仮定において行っている．よって提案手法はより標準モデルに近いモデルで Unforgeability を満たしているという利点があると言える．

5.2.2 理論的な計算コストとデータサイズ

Sanders らの構成 [7]，Katz らの構成 [8]，提案手法，および Bobolz らの構成 [5] の公開パラメータの生成と発行元の鍵生成の理論的な計算コストを表 1 に，証明書の作成と検証の計算コストを 2 に，ポリシー作成の計算コストを 3 に，ポリシー検証の計算コストを 4 に，証明生成

[*1] `https://github.com/Shigeo1042/Issuer-Hiding/tree/mthesis` にて実装を公開している

の計算コストを 5 に，証明検証の計算コストを 6 に示す．ただし，$R_{\mathbb{Z}_p}$ を $\mathbb{Z}_p$ 上の乱数生成，R_{G_1}, R_{G_2} をそれぞれ G_1, G_2 上の乱数生成，$E_{G_1}, E_{G_2}, E_{G_T}$ を G_1, G_2, G_T に対する $\mathbb{Z}_p$ 上の値の指数計算，$M_{G_1}, M_{G_2}, M_{G_T}$ をそれぞれ G_1, G_2, G_T 上の乗算，P をペアリング計算とする．一般的に，ペアリング計算，G_* に対する $\mathbb{Z}_p$ 上の指数計算，G_* 上の乗算の順に計算コストが高いとされている．

表 1　公開パラメータ生成と発行元の鍵生成の計算コストの比較

方式	公開パラメータ生成	発行元の鍵生成
Sanders ら [7]	$R_{\mathbb{Z}_p} + R_{G_1} + R_{G_2} + E_{G_1} + E_{G_2}$	$lR_{\mathbb{Z}_p} + lE_{G_2}$
Katz ら [8]	$(l+1)R_{G_1} + R_{G_2}$	$R_{\mathbb{Z}_p} + E_{G_2}$
提案手法	$(l+2)R_{G_1} + 2R_{G_2}$	$R_{\mathbb{Z}_p} + E_{G_2}$
Bobolz ら [5]	$(l+2)R_{G_1} + 2R_{G_2}$	$R_{\mathbb{Z}_p} + E_{G_2}$

表 1 から，公開パラメータの生成は Sanders らの構成が最も計算コストが高く，提案手法は Katz らの構成よりは G_1, G_2 要素の乱数生成が 1 回ずつ多いが，Bobolz らの構成と同じであることがわかる．属性数が増加するにつれて，Katz らの構成と提案手法，Bobolz らの構成の計算コストが増加する点に注意が必要である．発行元の鍵生成においては Katz らの構成，提案手法，Bobolz らの構成は同じ計算コストであり，Sanders らの構成が属性数に比例して計算コストが増加することがわかる．

表 2　証明書生成と検証の計算コストの比較

方式	証明書生成	証明書検証
Sanders ら [7]	$R_{\mathbb{Z}_p} + 3E_{G_1} + M_{G_1}$	$lE_{G_2} + lM_{G_2} + 2P$
Katz ら [8]	$R_{\mathbb{Z}_p} + (l+1)E_{G_1} + (l+2)M_{G_1}$	$(l+1)E_{G_1} + (l+1)M_{G_1} + M_{G_2} + 2P$
提案手法	$R_{\mathbb{Z}_p} + (l+1)E_{G_1} + (l+2)M_{G_1}$	$(l+1)E_{G_1} + (l+1)M_{G_1} + M_{G_2} + 2P$
Bobolz ら [5]	$R_{\mathbb{Z}_p} + (l+4)E_{G_1} + E_{G_2} + (l+1)M_{G_1}$	$lE_{G_1} + (l-1)M_{G_1} + 2M_{G_T} + 6P$

表 2 から，証明書生成と検証において，Katz らの構成と提案手法は同じ計算コストであり，Sanders らの構成が最も計算コストが低く，Bobolz らの構成が最も計算コストが高いことがわかる．なお，Sanders らの構成の証明書生成以外は属性数に比例して計算コストが増加する点に注意が必要である．

表 3 から，ポリシー作成において，提案手法は最も計算コストが低く，Katz らの構成が次に低いことがわかる．一方，Sanders らの構成が最も計算コストが高いことがわかる．

表 4 から，ポリシー検証において，Katz らの構成が最も計算コストが低く，次いで提案手法，Bobolz らの構成と計算コストが増えていき，Sanders らの構成が最も計算コストが高いことがわかる．

表 3　ポリシー作成の計算コストの比較

方式	ポリシー作成の計算コスト
Sanders ら [7]	$(2l+2)R_{\mathbb{Z}_p}+(4n_I\cdot l+2l+2)E_{G_2}+2n_I\cdot lM_{G_2}$
Katz ら [8]	$4R_{\mathbb{Z}_p}+(4n_I+2)E_{G_2}+2n_IM_{G_2}$
提案手法	$n_IR_{\mathbb{Z}_p}+n_IE_{G_2}+n_IM_{G_2}$
Bobolz ら [5]	$n_IR_{\mathbb{Z}_p}+n_IE_{G_1}+4n_IE_{G_2}+2n_IM_{G_1}$

表 4　ポリシー検証の計算コストの比較

方式	ポリシー検証の計算コスト
Sanders ら [7]	$(3n_I\cdot l+2l+2)E_{G_2}+(2n_I\cdot l+l+1)M_{G_2}$
Katz ら [8]	$(3n_I+2)E_{G_2}+(2n_I+1)M_{G_2}$
提案手法	$n_IE_{G_1}+n_IM_{G_1}+n_IM_{G_2}+(2n_I+2)P$
Bobolz ら [5]	$2n_IM_{G_T}+6n_IP$

表 5　証明生成の計算コストの比較

方式	証明生成の計算コスト
Sanders ら [7]	$(d'+2)R_{\mathbb{Z}_p}+3E_{G_1}+(l+1)E_{G_2}+d'E_{G_T}+M_{G_1}+(n_I(d'+1)-1)M_{G_2}+(d'-1)M_{G_T}+d'P$
[7] の水野の実装	$(d'+2)R_{\mathbb{Z}_p}+3E_{G_1}+(l+d'+1)E_{G_2}+M_{G_1}+(n_I(d'+1)+d'-2)M_{G_2}+P$
Katz ら [8]	$(d'+6)R_{\mathbb{Z}_p}+(l+d'+7)E_{G_1}+E_{G_2}+(l+d'+3)M_{G_1}+(n_I-1)M_{G_2}$
提案手法	$(d'+10)R_{\mathbb{Z}_p}+(l+d'+6)E_{G_1}+9E_{G_2}+(l+d'+3)M_{G_1}+4M_{G_2}$
[5] の水野の実装	$(d'+8)R_{\mathbb{Z}_p}+(d'+8)E_{G_1}+6E_{G_2}+(d'-1)M_{G_1}+4M_{G_T}+7P$
Bobolz ら [5]	$(d'+8)R_{\mathbb{Z}_p}+(d'+5)E_{G_1}+5E_{G_2}+5E_{G_T}+(d'-1)M_{G_1}+4M_{G_T}+7P$

表 5 において，Sanders らの構成は論文中で提示されている計算コストがあまりにも高いこと，簡単な最適化で大幅に計算コストが削減できることから，水野による実装の計算コストも示している．また，Bobolz らの構成においても，ark_bls12_381 では G_T 上の指数計算ができないため，水野による実装の計算コストも示している．証明生成において，提案手法よりも Katz らの構成の方が計算コストが低いことがわかる．一方，論文ベースの Sanders らの構成はペアリング計算が非開示属性数に比例して増加するため，最も計算コストが高いことがわかる．ただし，水野の実装では，ペアリング計算が 1 回に削減されており，この理論的な計算コストからだけでは比較が難しい．同様に Bobolz らの構成も指数計算こそ属性数 l に依存しないものの，ペアリング計算が 7 回と多いため，計算コストが高いことがわかるが，属性数 l が増加した場合の比較は難しい．

表 6　証明検証の計算コストの比較

方式	証明検証の計算コスト
Sanders ら [7]	$(2l-d'+2)E_{G_2}+(d'+1)E_{G_T}+(n_I\cdot l-n_I+2)M_{G_2}+(d'+1)M_{G_T}+(d'+2)P$
[7] の水野の実装	$(2l+3)E_{G_2}+(n_I\cdot l+d'+1)M_{G_2}+2M_{G_T}+3P$
Katz ら [8]	$(l+5)E_{G_1}+2E_{G_2}+(l+3)M_{G_1}+(n_I+1)M_{G_2}+2P$
提案手法	$(l+5)E_{G_1}+6E_{G_2}+(l+3)M_{G_1}+4M_{G_2}+4P$
[5] の水野の実装	$(l+6)E_{G_1}+E_{G_2}+(l-1)M_{G_1}+7M_{G_T}+12P$
Bobolz ら [5]	$lE_{G_1}+8E_{G_T}+(l-1)M_{G_1}+7M_{G_T}+12P$

表 6 においても，Sanders らの構成は論文中で提示されている計算コストがあまりにも高いこと，簡単な最適化で大幅に計算コストが削減できることから，水野による実装の計算コストも示している．また，Bobolz らの構成においても，ark_bls12_381 では G_T 上の指数計算ができないため，水野による実装の計算コストも示している．証明検証においては Katz らの構成が最も計算コストが低く，次いで提案手法であることがわかる．Bobolz らの構成はペアリング計算が 12 回と多いため，計算コストが高いことがわかるが，Sanders らの構成の水野の実装では約 l 回分指数計算が多く，G_2 上の乗算も $n_I\cdot l$ と属性数と検証者が受け入れ可能な発行元数に比例して増加するため，属性数 l や検証者が受け入れ可能な発行元数 n_I が増加した場合の比較は難しい．

次に，表 7 と 8 に，Sanders らの構成 [7]，Katz らの構成 [8]，提案手法，および Bobolz らの構成 [5] の各アルゴリズムにおけるデータサイズを示す．ただし，$|\mathbb{Z}_p|, |G_1|, |G_2|$ をそれぞれ $\mathbb{Z}_p$ or $\mathbb{Z}_p^*, G_1, G_2$ の要素 1 つ分のサイズとする．

表 7　各方式の公開パラメータ，発行元の鍵，証明書のデータサイズ比較

方式	公開パラメータ	発行元の鍵	証明書
Sanders ら [7]	$2\|G_1\|+2\|G_2\|$	$l\|\mathbb{Z}_p\|+l\|G_2\|$	$2\|G_1\|$
Katz ら [8]	$(l+1)\|G_1\|+\|G_2\|$	$\|\mathbb{Z}_p\|+\|G_2\|$	$\|\mathbb{Z}_p\|+\|G_1\|$
提案手法	$(l+2)\|G_1\|+2\|G_2\|$	$\|\mathbb{Z}_p\|+\|G_2\|$	$\|\mathbb{Z}_p\|+\|G_1\|$
Bobolz ら [5]	$(l+2)\|G_1\|+2\|G_2\|$	$\|\mathbb{Z}_p\|+\|G_2\|$	$2\|G_1\|+\|G_2\|$

表 7 から，公開パラメータのデータサイズは Sanders らの構成が最も小さく，Katz らの構成，提案手法と Bobolz らの構成と大きくなっていくことがわかる．発行元の鍵のデータサイズは Sanders らの構成のみ属性数に比例して増加するが，その他の方式は同じデータサイズであることがわかる．証明書のデータサイズは Katz らの構成と提案手法が同じであり，最も小さいことがわかる．一方で，Bobolz らの構成が最も大きいことがわかる．

表 8　各方式のポリシーと提示トークンのデータサイズ比較

方式	ポリシー	提示トークン
Sanders ら [7]	$(l+2)\lvert\mathbb{Z}_p\rvert + (n_I \cdot l + l + 1)\lvert G_2\rvert$	$(d'+1)\lvert\mathbb{Z}_p\rvert + 2\lvert G_1\rvert + \lvert G_2\rvert$
Katz ら [8]	$3\lvert\mathbb{Z}_p\rvert + (2n_I+1)\lvert G_2\rvert$	$(d'+4)\lvert\mathbb{Z}_p\rvert + 3\lvert G_1\rvert + \lvert G_2\rvert$
提案手法	$n_I\lvert\mathbb{Z}_p\rvert + \lvert G_1\rvert + (2n_I+1)\lvert G_2\rvert$	$(d'+7)\lvert\mathbb{Z}_p\rvert + 3\lvert G_1\rvert + 4\lvert G_2\rvert$
Bobolz ら [5]	$(n_I+1)\lvert G_1\rvert + (2n_I+1)\lvert G_2\rvert$	$(d'+7)\lvert\mathbb{Z}_p\rvert + 7\lvert G_1\rvert + 6\lvert G_2\rvert$

表 8 から，ポリシーのデータサイズは Katz らの構成が最も小さく，次いで提案手法，Bobolz らの構成，Sanders らの構成の順に大きくなることがわかる．ただし，Sanders らの構成は属性数 l と検証者が受け入れ可能な発行元数 n_I に比例してポリシーのデータサイズが増加する点に注意が必要である．提示トークンのデータサイズは Sanders らの構成が最も小さく，次いで Katz らの構成，提案手法，Bobolz らの構成の順に大きくなることがわかる．

5.2.3　ark_bls12_381 のベンチマーク

使用した ark_bls12_381 での $R_{\mathbb{Z}_p}, R_{G_1}, R_{G_2}, E_{G_1}, E_{G_2}, E_{G_T}, M_{G_1}, M_{G_2}, M_{G_T}, P$ をそれぞれ 1000 回ずつ実行した時間を以下の表 9 に示す．なお，実際には Vec 型の操作も含んでいるため，オーダーの比較程度に見るのが良いだろう．

表 9　ark_bls12_381 での各処理 1000 回分の実行時間

処理	実行時間 [ms]
$R_{\mathbb{Z}_p}$	3.6384×10^{-2}
R_{G_1}	98.940
R_{G_2}	774.10
E_{G_1}	122.06
E_{G_2}	345.24
E_{G_T}	542.15
$M_{\mathbb{Z}_p}$	8.5940×10^{-7}
M_{G_1}	0.37797
M_{G_2}	1.1167
M_{G_T}	2.7322
P	861.27

この表から実行時間は $P > E_{G_*} > R_{G_*} > M_{G_*}$ であること，G_* の中だと $G_T > G_2 > G_1$ の順に実行時間が長いこと，$R_{\mathbb{Z}_p}$ の実行時間は非常に短いことがわかる．また，R_{G_2} のみ E_{G_2} より長く，$E_{G_*} > R_{G_*}$ が成り立っていない．また，計算コストの理論値では省略しているが，

$\mathbb{Z}_p$ 上の値の乗算は非常に高速であることがわかる．そのため，$(h_i^{m_i})^c$ のような計算は先に $m_i \cdot c$ を求めてから h_i の肩に乗せた方が効率が良いことがわかる．

5.2.4　実行時間

Sanders らの構成 [7]，Katz らの構成 [8]，提案手法，および Bobolz らの構成 [5] の公開パラメータの生成と発行元の鍵生成の実行時間の比較を表 10 に，証明書作成の実行時間の比較を図 6 に，証明書検証の実行時間の比較を図 7 に，ポリシー作成時間の比較を図 8 に，ポリシー検証時間の比較を図 9 に，検証者が受け入れ可能な発行元数 $n_I = 1000$ に固定し，属性数 l を変化させた時の証明生成の実行時間の比較を図 10 に，$l = 50$ に固定し，n_I を変化させた時の証明生成の実行時間の比較を図 11 に，$n_I = 1000$ に固定し，l を変化させた時の証明検証の実行時間の比較を図 12 に，$l = 50$ に固定し，n_I を変化させた時の証明検証の実行時間の比較を図 13 に示す．なお，図 8 から図 13 まで，Sanders らの構成の実行時間を含んだ図を左に，Sanders らの構成の実行時間を除いた図を右に示している．また，図 8 と図 9 の左図のみ両対数グラフになっている．

表 10　公開パラメータの生成と発行元の鍵生成の実行時間の比較

方式	公開パラメータ生成 [ms]	発行元の鍵生成 [μs]
Sanders ら [7]	21.854×10^{-3}	17.328×10^{3}
Katz ら [8]	8.3415	354.05
提案手法	8.9179	354.19
Bobolz ら [5]	8.9141	353.78

表 10 から，公開パラメータ生成と発行元の鍵生成において，表 1 に示した通りの結果が得られた．なお，それぞれの方式で理論上の計算コストは同じでも差があるが，これは誤差であると言えるだろう．

図 6 から，証明書生成において，Sanders らの構成が最も高速であり，次いで提案手法と Katz らの構成，Bobolz らの構成の順に遅くなることがわかる．ただし，証明書生成は単なる PS 署名，BBS 署名，Groth 署名の生成であるため，Issuer-Hiding AC の特性とは無関係である点に注意が必要である．実際，提案手法と Katz らの構成はどちらも BBS 署名を用いているため，ほぼ同じ実行時間となっている．

図 7 から，証明書検証においては提案手法と Katz らの構成がほぼ同じ実行時間であり，最も高速であることがわかる．属性数 $l = 15$ あたりまでは Sanders らの構成が次に高速であるが，属性数が増加するにつれて実行時間が急激に増加し，最も遅くなることがわかる．また，全体的に証明書検証は証明書生成よりも遅いことがわかる．

図 8 から，ポリシー作成において，提案手法が最も高速であり，次いで Katz らの構成，Bobolz らの構成，Sanders らの構成の順に遅くなることがわかる．特に Sanders らの構成は両対数グラ

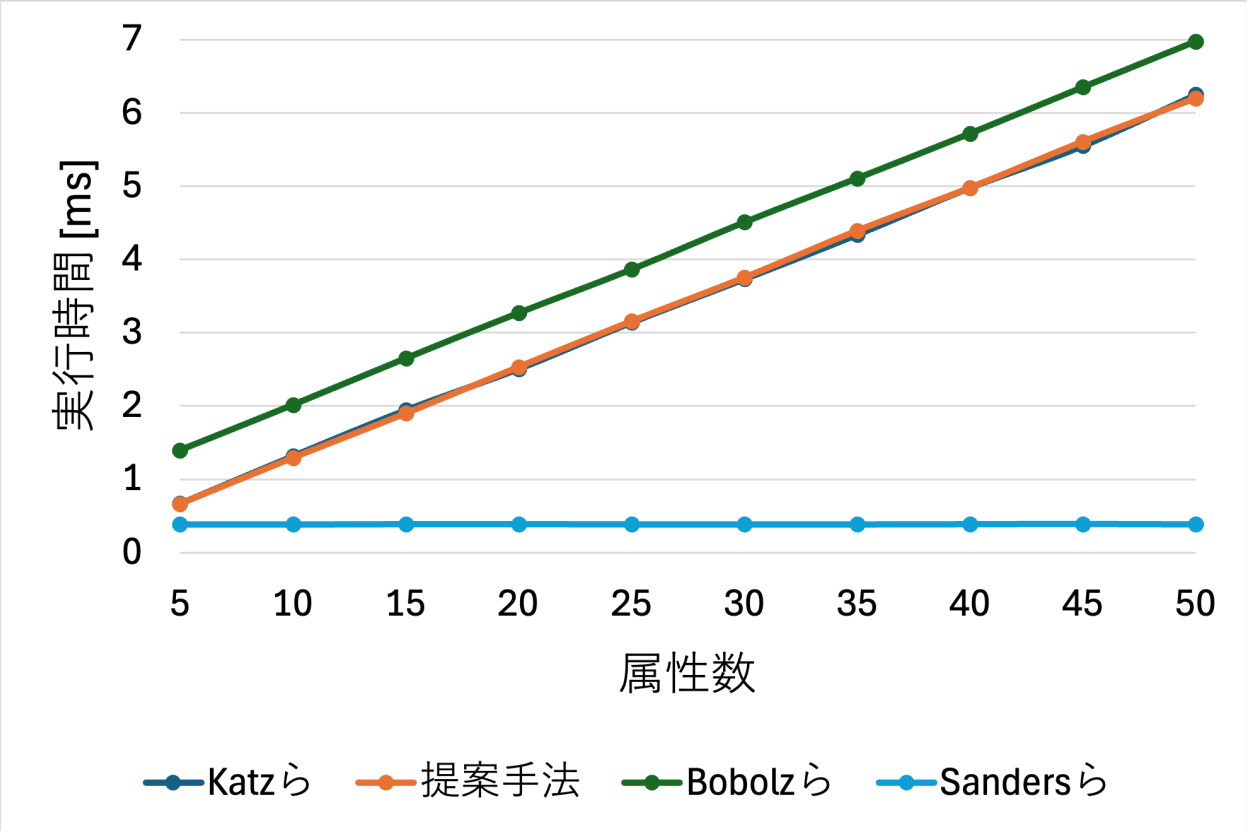

図 6　証明書生成時間の比較

フで見ても他の 3 方式と比べて圧倒的に遅いことがわかる．

図 9 から，ポリシー検証において，Katz らの構成が最も高速であり，次いで提案手法，Bobolz らの構成，Sanders らの構成の順に遅くなることがわかる．こちらでも Sanders らの構成は両対数グラフで見ても他の 3 方式と比べて圧倒的に遅いことがわかる．また，全体的にポリシー検証時間はポリシー作成時間よりも遅いことがわかる．

図 10 から，検証者が受け入れ可能な発行元数を $n_I = 1000$ に固定し，属性数を変えた時の証明生成において，Katz らの構成が最も高速であり，次いで属性数が少ない時は提案手法，属性数が多い時は Bobolz らの構成が続き，Sanders らの構成が最も遅いことがわかる．提案手法と Bobolz らの構成は属性数が 45 あたりで逆転していることがわかる．また，どの方式も非開示属性数が増加するにつれて証明生成時間が増加していることがわかる．また，Bobolz らの構成のみ，証明生成時間が属性数が増えるにつれて低下している箇所があることがわかる．

図 11 から，属性数を $l = 50$ に固定し，検証者が受け入れ可能な発行元数を変えた時の証明生成において，Katz らの構成が最も高速であり，次いで Bobolz らの構成，提案手法，Sanders らの構成の順に遅くなることがわかる．

図 12 から，検証者が受け入れ可能な発行元数を $n_I = 1000$ に固定し，属性数を変えた時の証明検証において，Katz らの構成が最も高速であり，次いで提案手法，Bobolz らの構成，Sanders らの構成の順に遅くなることがわかる．また，Sanders らの構成以外は開示属性数が増加するに

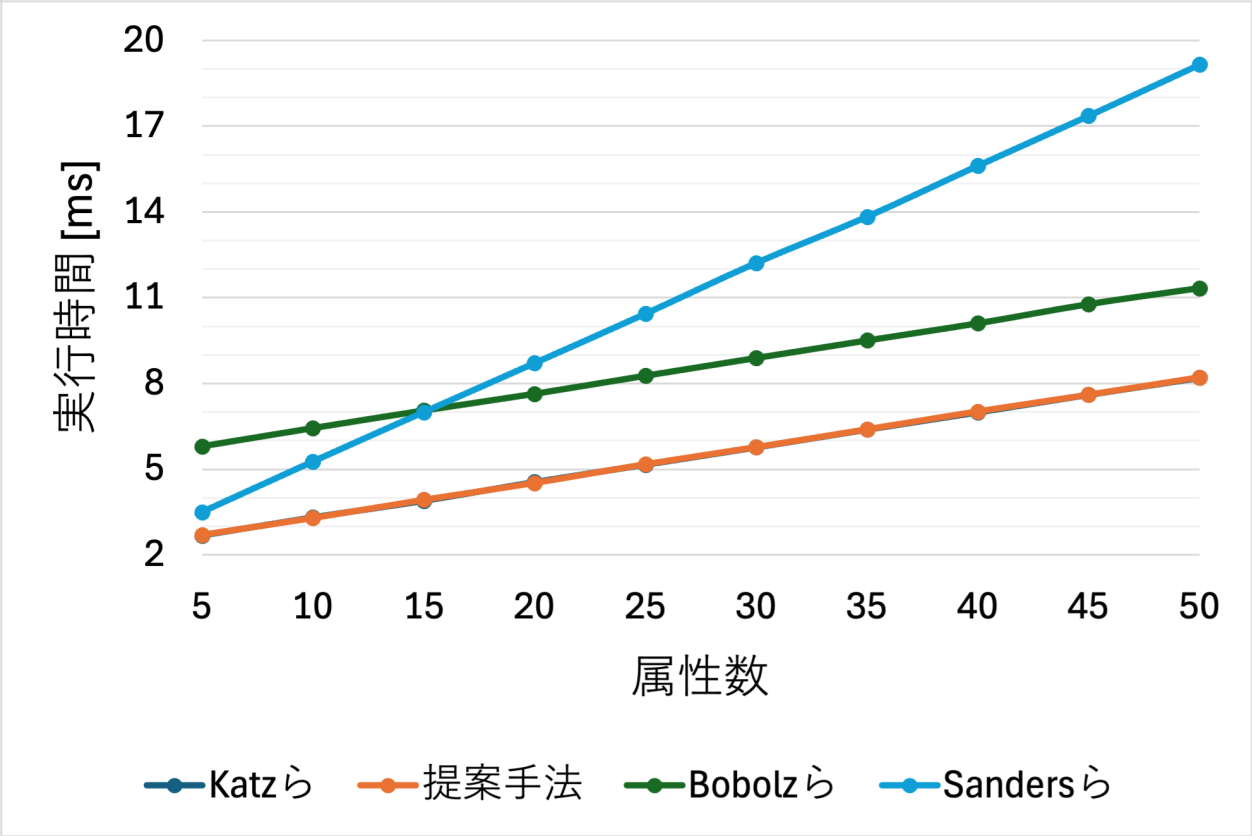

図 7　証明書検証時間の比較

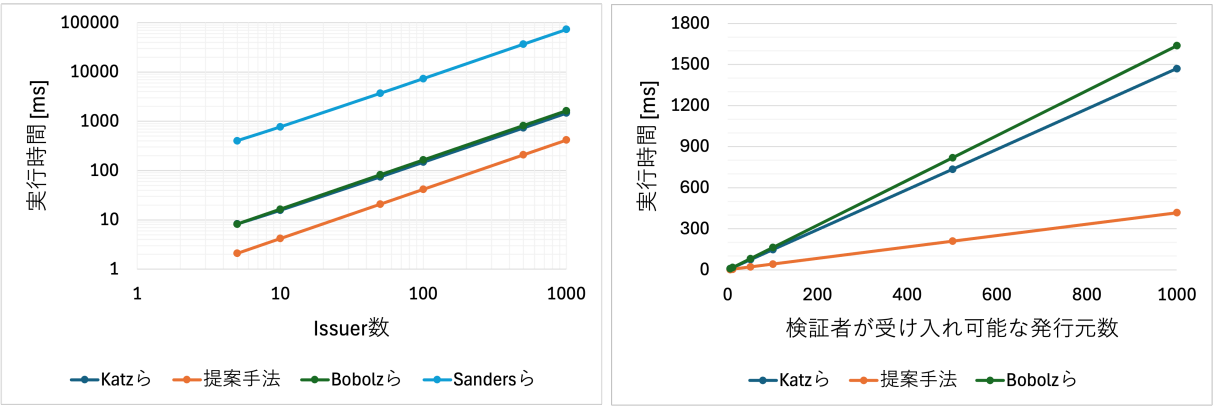

図 8　ポリシー作成時間の比較

つれて証明検証時間が増加していることがわかる．

図 13 から，属性数を $l = 50$ に固定し，検証者が受け入れ可能な発行元数を変えた時の証明検証において，Katz らの構成が最も高速であり，次いで提案手法，Bobolz らの構成，Sanders らの構成の順に遅くなることがわかる．

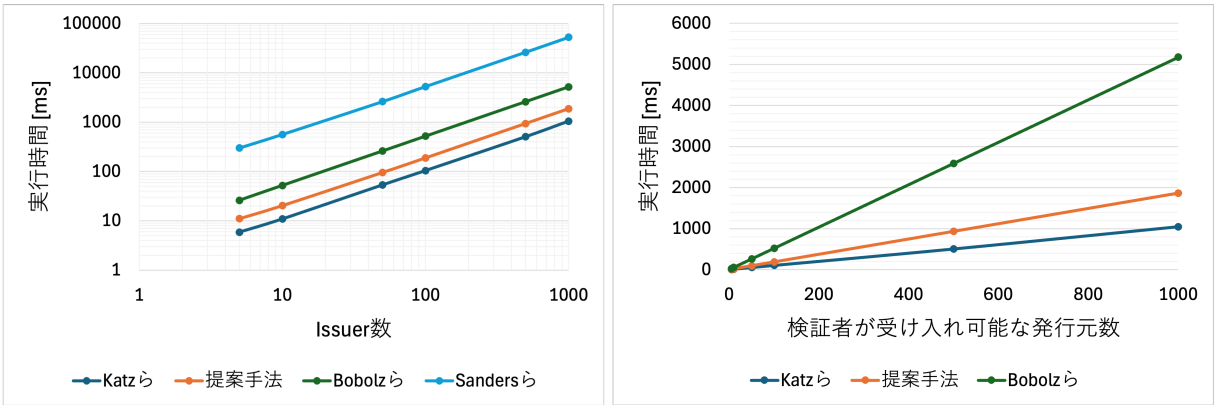

図 9　ポリシー検証時間の比較

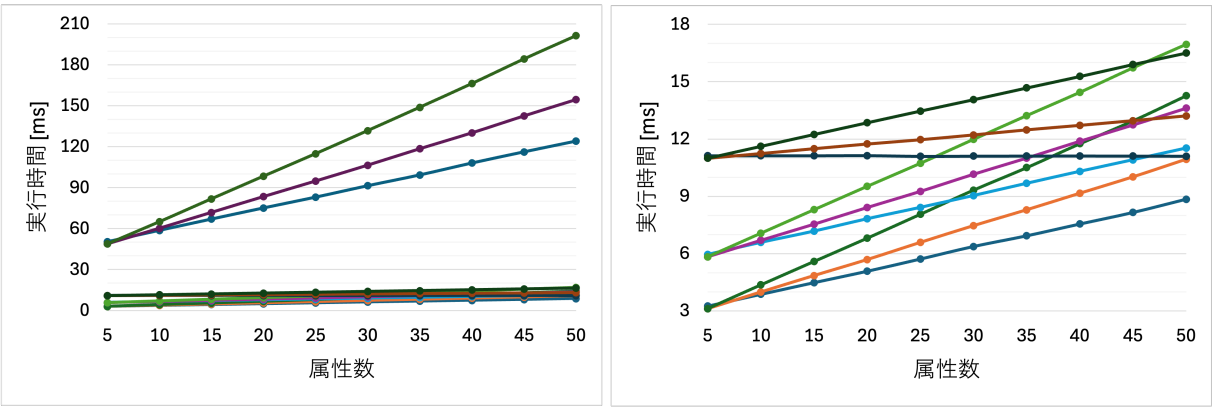

図 10　属性数を変えた時の証明生成時間の比較

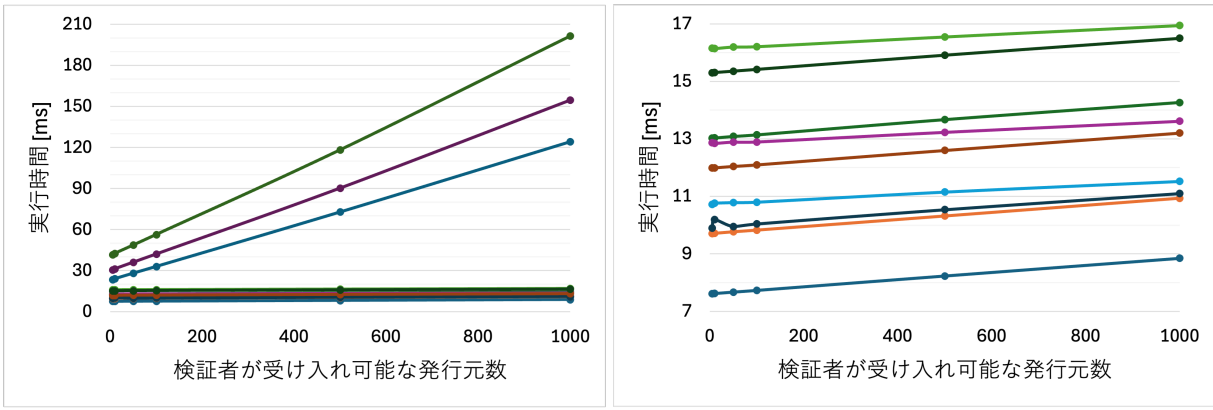

図 11　検証者が受け入れ可能な発行元数を変えた時の証明生成時間の比較

5.3　考察

5.3.1　公開パラメータ生成と鍵生成

公開パラメータ生成と鍵生成は全ての方式において一度きりの処理であり，Katz らの構成では，BBS 署名のままであるが，提案手法では，BBS 署名の公開パラメータに加えて G_1, G_2 の

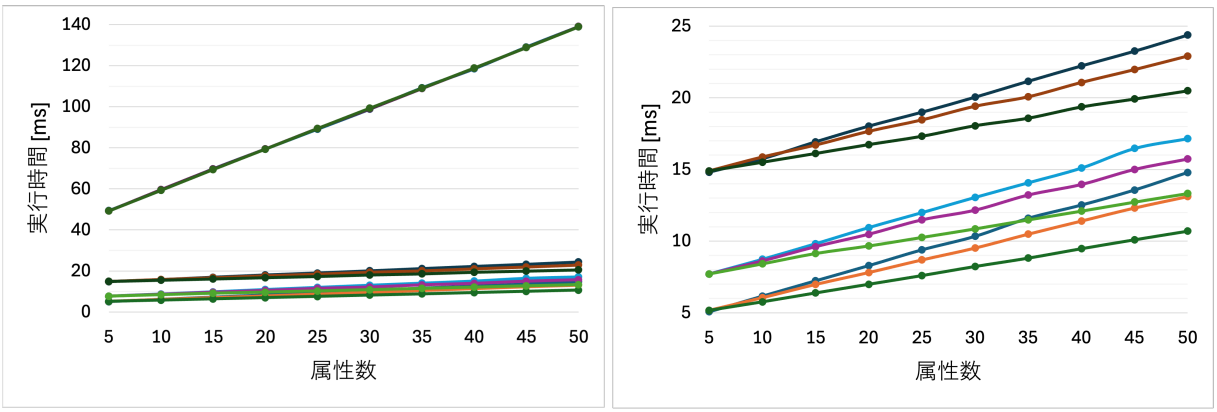

図 12　属性数を変えた時の証明検証時間の比較

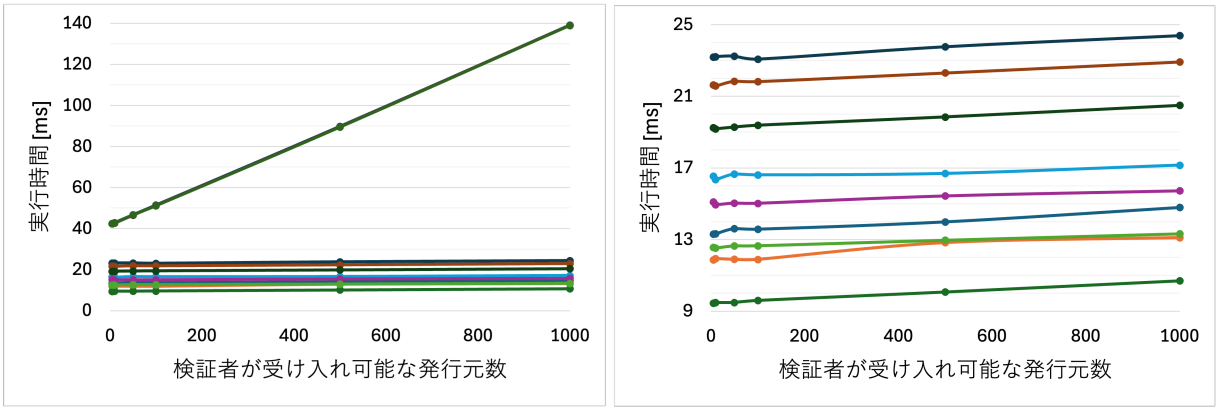

図 13　検証者が受け入れ可能な発行元数を変えた時の証明検証時間の比較

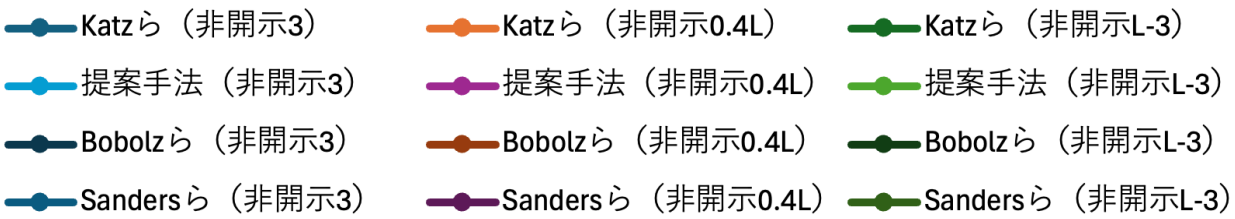

図 14　凡例

要素を 1 つずつ追加しているため，わずかに計算コストが増加している．一方で，発行元の鍵生成においてはどちらの手法も BBS 署名の鍵生成と同じであるため，ほぼ同じ実行時間となっている．Sanders らの構成では，公開パラメータは少なく，発行元の鍵生成は属性数に比例して計算コストが増加するため，属性数が増加するにつれて実行時間も増加していることがわかる．Bobolz らの構成では，発行元の公開鍵が BBS 署名の公開鍵と同じであるため，発行元の鍵生成はほぼ同じ実行時間となっている．

ただし，Sanders らの鍵生成および，その他 3 つの方式の公開パラメータ生成において，属性数 l は実際には固定されることが多い．というのも，属性数が変わるごとに鍵生成や公開パ

ラメータ生成を行うのは非現実的である他，特に BBS 署名においては標準化が進んでいる関係で，公開パラメータに含まれる h_i が全ての発行元で共通になることが多いことが理由である．よって実装時には $l = 50$ に固定している．どの程度の属性数が現実的かは利用シーンによるが，一般的には数十程度であると考えられる．

公開パラメータの生成と発行元の鍵生成が一回ずつ行えば良いこと，どちらも発行元が実行することを踏まえると，公開パラメータ生成と発行元の鍵生成を同時に行うことが多いと考えられる．すると公開パラメータ生成と発行元の鍵生成の実行時間の合計を見れば実際の発行元のセットアップにかかる時間がわかると言える．表 10 より，Sanders らの方式はセットアップに約 17 ミリ秒かかり，最も時間がかかる．他の 3 方式は約 9 ミリ秒であることを考えると Sanders らの方式は劣っていると言える．表 9 よりこれは，R_{G_1} よりも E_{G_2} の方が計算コストが高いためであると考えられる．ただし，17 ミリ秒も十分高速なため実用的には問題はない．

5.3.2　証明書の作成と検証

全ての方式において，証明書作成と検証は単なる署名生成と署名検証であるため，Issuer-Hiding AC の特性とは無関係である．よって，Sanders らの構成は PS 署名の署名生成と同じであり，指数計算が少なく，属性数に比例しないため，最も高速かつ署名生成時間がミリ秒のオーダーだとほぼ一定となる．一方で，署名検証は属性数に比例して計算コストが増加するため，属性数が増加するにつれて署名検証時間も増加する．また，署名検証は他の 3 方式と異なり，G_2 要素で行われる．表 9 で示した通り，G_1 要素の演算より G_2 要素の演算の方が処理時間が長く，G_2 要素の指数演算とペアリング演算の処理時間は 2 から 3 倍程度しか変わらないため，属性数が増えるに従い，実行時間は最も遅くなる．提案手法と Katz らの構成は BBS 署名の署名生成・検証と同じであるため，ほぼ同じ実行時間となっており，属性数に比例して署名生成・検証時間が増加する．Bobolz らの構成は Groth 署名の署名生成・署名検証と同じであり，署名生成は指数計算が多いため最も遅くなる．一方で，署名検証はペアリング演算が多いため，属性数が少ないうちは最も遅いが，Sanders らの構成が G_2 要素の指数演算が多いため，属性数が 15 の時あたりで Sanders らの構成の方が遅くなる．

5.3.3　ポリシーの作成と検証

表 3 と表 4 から，図 8 と図 9 の実行時間の計測結果と一致して，提案手法はポリシー作成において最も計算コストが低く，ポリシー検証においては Katz らの構成に次いで 2 番目に計算コストが低いことがわかる．Sanders らの構成では，ポリシー作成と検証の両方において属性数と発行元数に比例して計算コストが増加するため，最も計算コストが高くなることがわかる．ただし，実装では，属性数 l を 50 に固定している．というのも，属性数が変わるごとに鍵生成するのは非現実的であり，実際の利用時にはある程度大きい数値に固定し，必要に応じて部分的に鍵を利用することが考えられるためである．よって実行時間の計測時の，理論的なポリシー作成の計算コストは $102R_{\mathbb{Z}_p} + (200n_I + 102)E_{G_2} + 100n_I M_{G_2}$，ポリシー検証の計算コストは

$(150n_I + 102)E_{G_2} + (100n_I + 51)M_{G_2}$ である．よって図 8 と図 9 で Sanders らの構成が他の方式と比べて圧倒的に遅いのは，他の方式と比較して約 50 倍の計算コストがかかっているためであり，それはグラフからも確認できる．

5.3.4　証明の生成と検証

表 5 と図 10 から，Sanders らの構成はやはり，検証者が受け入れ可能な発行元数 n_I に比例してペアリング計算が増加するため，最も計算コストが高く，実行時間も最も遅いことがわかる．また，Bobolz らの構成では指数計算と G_1 上の要素の乗算において l に比例しないため，属性数が 50 程度になってくるとペアリング計算に 7 回分の差があれど提案手法よりも高速になることがわかる．一方で，Katz らの構成と比較したときに $l - 7$ 回分の指数計算の差があり，こちらは属性数が 50 程度だとペアリング計算 7 回分を埋め合わせられないこともわかる．

表 5 から，検証者が受け入れ可能な発行元数 n_I を変えた時の証明生成において，提案手法と Bobolz らの構成は検証者が受け入れ可能な発行者数 n_I に依存していないはずであるが，図 11 ではわずかに増加していることがわかる．これは，実際の発行元の公開鍵への検証者の署名を探す処理に検証者が受け入れ可能な発行元数 n_I が影響しているためであると考えられる．実装では，発行元の公開鍵への署名をリストから線形探索で探しており，かつ，実際の発行元の公開鍵がリストの中のどこにあるかを実行時間から推測されないように署名を見つけてもリスト全てにアクセスしているため，検証者が受け入れ可能な発行元数 n_I に比例してわずかに実行時間が増加していると考えられる．ただし，これはリストのデータ構造を工夫することで，検証者が受け入れ可能な発行元数 n_I に依存しないようにできると考えられる．もう一つ考えられる原因として，チャレンジ c の生成において，ハッシュ関数に入力するポリシーのサイズが検証者が受け入れ可能な発行元数 n_I に比例して増加するため，わずかに実行時間が増加している可能性もある．こちらは論文でポリシーを含めるようにしているため，検証者が受け入れ可能な発行元数 n_I に依存しないようにするのは難しいと考えられる．ただし，どちらも非常にわずかな増加であり，実際の利用時には無視して良いレベルであると考えられる．

表 6 と図 12 から，検証者が受け入れ可能な発行元数 n_I に比例して計算コストが増加する Sanders らの構成は最も計算コストが高く，実行時間も最も遅いことがわかる．また，その他の構成についても順当にペアリング計算回数が多いほど実行時間が遅くなっていることがわかる．表 6 から，本来は非開示属性数に依存しないはずの Sanders らの構成以外の方式が，図 12 と図 13 で差が出ていることがわかる．考えられる原因として，チャレンジ c の生成において，ハッシュ関数に入力する属性の数が開示属性数に比例して増加するため，実行時間が増加している可能性がある．一方で，Sanders らの構成については G_2 上の乗算が d' に比例して増加するため，非開示属性数に比例して実行時間が増加するはずであるが，図 12 ではあまり増加していないことがわかる．これは，表 9 より M_{G_2} の 1 回あたりの実行時間が 1.1167 マイクロ秒であり，d' は高々 50 程度の差しかないことを考えると，高々 0.055 ミリ秒の差しか生まれず，目視では現れない程度の差にしかならないためであると考えられる．また，検証者が受け入れ可能

な発行元数 n_I についても同様に，チャレンジ c の生成において，ハッシュ関数に入力するポリシーのサイズが検証者が受け入れ可能な発行元数 n_I に比例して増加するため，実行時間が増加している．

5.3.5　データサイズ

ark_bls12_381 において，$|\mathbb{Z}_p| = 32$ バイト，$|G_1| = 104$ バイト，$|G_2| = 200$ バイトである．

表 7 に示した公開パラメータのデータサイズと発行元の鍵のデータサイズから，公開パラメータのデータサイズを属性数 l に比例させると鍵サイズが定数に，発行元の鍵サイズを属性数 l に比例させると公開パラメータサイズが定数になることがわかる．

表 8 に示したデータサイズから，提案手法は [8] に比べて，ポリシーサイズが大きく，証明サイズも大きいことがわかる．ポリシーサイズが大きくなる原因として，n_I 個の乱数 e を作成していることが挙げられる．実際の利用時において，$i, j \in [n_I], i \neq j$ の時でも $e_i = e_j$ としてよければ，ポリシーサイズを削減することは可能であるが，セキュリティへの影響についてはよく検討する必要があり，今後の課題としたい．

また，Sanders らの構成や Katz らの構成と比べて提案手法や Bobolz らの構成が証明サイズが大きくなる原因として，Bobolz らの構成と提案手法では，ポリシーに関する証明に利用する値を追加で含める必要があることが挙げられる．

5.4　まとめ

安全性証明に用いた計算モデルは，提案手法が最も現実に近いものであると言える．また，計算コストに関しても，提案手法はポリシー作成については最も計算コストが低く，ポリシー検証や証明検証についても Katz らの構成に次いで 2 番目に計算コストが低いことがわかった．また，データサイズは概ね Katz らの構成が最も小さく，提示トークンこそ Sanders らの構成が最も小さいものの，提案手法も比較的小さいと言える．ただし，実際には証明書の検証，ポリシー検証，証明生成はスマホなどのリソースが限られた環境で行われることが多いと考えられるため，計算コストが低いことが望ましい．

6 Verifiable Credentials

6.1 Verifiable Credentials の概要

Verifiable Credentials (VC) は World Wide Web Consortium (W3C) によって標準化が進められているデジタル証明書のフレームワークである [1]．以下では，VC について Verifiable Credentials Data Model 1.0[1] に基づいて説明する．

6.1.1 データモデル

Verifiable Credential　クレデンシャルは Issuer が作成した 1 つ以上の Holder の属性からなる．このクレデンシャルを暗号学的に検証可能にしたものが Verifiable Credential である．VC はメタデータと 1 つ以上の属性と属性に対しての Issuer のデジタル署名から構成される．VC ではメタデータから Issuer が誰かわかり，Issuer の公開鍵を用いて署名の検証ができる．

Verifiable Presentation　特定の Verifier と共有される，1 つ以上の Issuer から発行された 1 つ以上の VC から作成されるデータのこと．Holder が VC を Verifier に提示する際に，Verifiable Presentation (VP) を作成し，提示する．

ゼロ知識証明などを用いて VP の元となる VC の属性の一部を選択的に開示することができる．

6.1.2 VC の登場人物

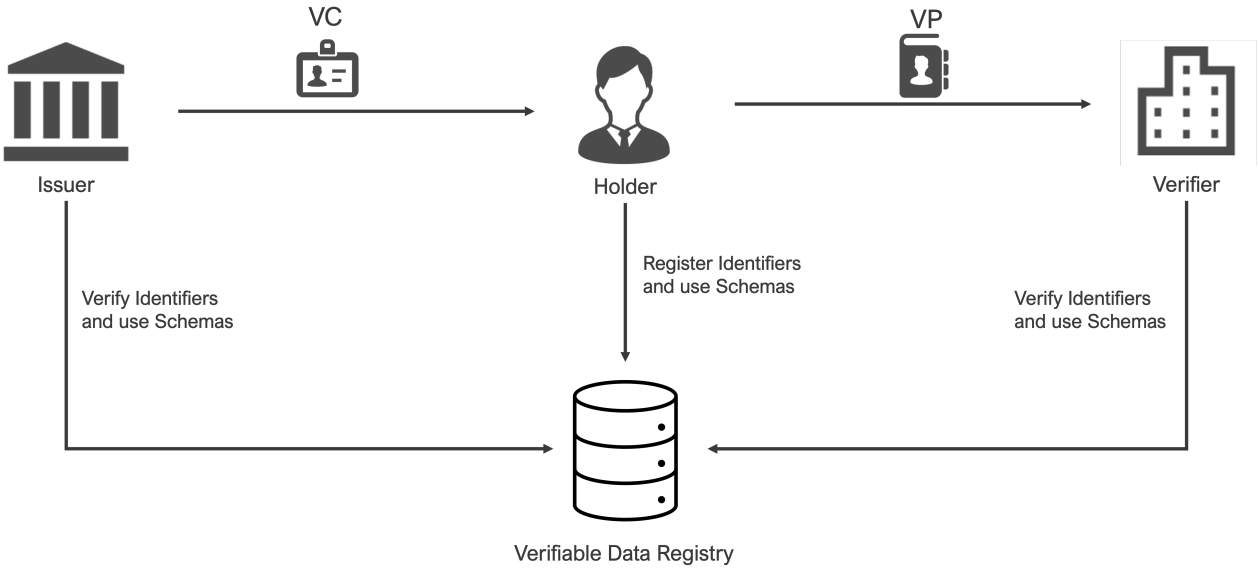

図 15　Verifiable Credentials のシステム全体の概要図

Verifiable Credentials のシステム全体の概要と，登場人物，およびそれぞれの役割について示

す．図 15 にシステムの概要図を示す．

Holder　1 つ以上の VC を持っており，保有している VC から VP を生成する．

Issuer　1 つ以上の Holder の属性に対して署名を行うことで VC を生成し，Holder に発行する．

Verifier　Holder から提示された VP を検証する．検証のために使用する Issuer の公開鍵は Verifiable Data Registry から取得する．

Verifiable Data Registry　VC の発行者の公開鍵や，失効リストなど VC の検証に必要な情報を管理する分散型台帳やデータベースのこと．Verifiable Data Registry 内の情報は正しいとみなして利用されるため，改ざんできないことや，誰に管理されているか正しく記録されていることなど，信頼性が求められる．

6.2　選択的開示

VC に記載されている属性のうち，必要な属性のみを選択的に開示して VP を作成し，Verifier に提示しても Verifier が VP を検証できることを選択的開示と呼ぶ．選択的開示には連結不可能な選択的開示と連結可能な選択的開示の 2 種類がある．

6.2.1　連結不可能な選択的開示

連結不可能な選択的開示とは，ある Holder が同じ属性を複数回選択的に開示したとしても，暗号学的にはそれらの提示が同一の Holder からのものであるかどうかを，Verifier が判別できない選択的開示のことを指す．主にゼロ知識証明を用いて実現される．

ただし，暗号学的に連結不可能な選択的開示を実現していても，マイナンバーのように属性自体から Holder を一意に特定できる場合は，連結することが可能であるため注意が必要である．

Anonymous Credentials (AC) における選択的開示は連結不可能な選択的開示である．中でも，CL 署名（Camenisch-Lysyanskaya 署名）[31] や PS 署名（Pointchavel-Sanders 署名）[6]，BBS 署名 [2] などが VC において用いられることが議論されている．

6.2.2　連結可能な選択的開示

連結可能な選択的開示とは，ある Holder が同一の VC から異なる属性を選択的に開示した場合に，Verifier がそれらの提示（VP）が同一の VC からのものであるかどうかを判別できる選択的開示のことを指す．IETF では，選択的開示を可能とする署名データフォーマットとして SD-JWT (Selective Disclosure JSON Web Token) [32] が策定されている．SD-JWT では，Issuer が属性ごとにソルト付きハッシュ関数でハッシュ値を計算し，そのハッシュ値の集合に対して署名を行なっている．Holder は，VP を作成する際に，開示したい属性はそのまま提示し，開示しない属性はハッシュ値のみを提示することで，Verifier はハッシュ値を計算し，署名の検証を

行うことで選択的開示を実現している．この方式では，Issuer が署名したハッシュ値の集合は異なる提示間で共通であるため，Verifier は同一の VC からの提示であることを判別できる．

連結不可能性を実現するために，あらかじめ大量に署名を用意しておき，毎提示ごとに異なる署名を利用する方法や，開示する属性が異なる場合は異なるソルト値を用いてハッシュ値を計算し署名を行う方法などが考えられるが，いずれも効率的ではない．

6.3 Issuer-Hiding Verifiable Credentials の検討

3 章と 4 章で説明した Issuer-Hiding AC は，あくまでも Anonymous Credentials であり，VC のデータフォーマットやシステム全体に則っているわけではない．特に，VC における Issuer の情報は，VC のメタデータに含まれており，Issuer-Hiding AC をそのまま VC に適用することはできない．そこで，本節では Issuer-Hiding AC を VC に適用するための検討を行う．具体的には，Verifier が発行する VC のデータフォーマットと Issuer-Hiding VP のフォーマットについて提案を行う．

6.3.1 Issuer-Hiding Verifiable Credentials のフォーマットの提案

VC のフォーマット　まずリスト 1 に Issuer が発行した学生証 VC の例を示す．なお，23 行目の cryptosuite は使用する方式により変更が必要である．

リスト 1　学生証 VC の例

```
 1 {
 2   "@context": [
 3     "https://www.w3.org/2018/credentials/v1",
 4     "https://www.w3.org/ns/data-integrity/v1",
 5     "https://schema.org/"
 6   ],
 7   "id": "http://issuer1.example.org/cred1",
 8   "type": [ "VerifiableCredential", "StudentCredential" ],
 9   "issuer": "did:example:issuer1",
10   "issuanceDate": "2023-04-01T00:00:00Z",
11   "expirationDate": "2026-03-31T00:00:00Z",
12   "credentialSubject": {
13     "id": "http://issuer1.example.org/s1",
14     "type": "Person",
15     "givenName": "John",
16     "familyName": "Smith",
17     "birthDate": "2004-01-01",
18     "award": "Awesome Student"
19   },
20   "proof": {
21     "type": "DataIntegrityProof",
22     "cryptosuite": "bbs-2023",
23     "verificationMethod": "did:example:issuer1#key-1",
24     "proofPurpose": "assertionMethod",
25     "created": "2023-01-01T00:00:00Z",
26     "proofValue": "uq2OqMazydSY..."
27   }
28 }
```

次に，Verifier が信頼できる Issuer の公開鍵を Holder に提示するための VC の例をリスト 2 およびリスト 3 に示す．後に示す VP を作成するにあたり，より現行の VP の仕様に適合しやすいと考えているため，一つの Issuer に対して一つの VC を作成している．なお，どちらの VC も 19 行目の cryptosuite は使用する方式により変更が必要である．

リスト 2 Verifier の作成した VC の例 1

```
1  {
2    "@context": [
3      "https://www.w3.org/2018/credentials/v1",
4      "https://www.w3.org/ns/data-integrity/v1",
5      "https://schema.org/"
6    ],
7    "type": [ "VerifiableCredential", "TrustedIssuerCredential" ],
8    "issuer": "did:web:verifier.example.org",
9    "issuanceDate": "2024-01-01T00:00:00Z",
10   "expirationDate": "2026-01-01T00:00:00Z",
11   "credentialSubject": {
12     "id": "did:example:issuer1",
13     "type": "TrustedIssuer",
14     "verificationMethod": "did:key:zUC7HaSjNELSGG..." // Issuer1 の公開鍵
15   },
16   "proof": {
17     "type": "DataIntegrityProof",
18     "cryptosuite": "bbs-2023",
19     "verificationMethod": "did:web:verifier.example.org#key-1",
20     "proofPurpose": "assertionMethod",
21     "created": "2024-01-01T00:00:00Z",
22     "proofValue": "uqaTINcX80a..."
23   }
24 }
```

リスト 3 Verifier の作成した VC の例 2

```
1  {
2    "@context": [
3      "https://www.w3.org/2018/credentials/v1",
4      "https://www.w3.org/ns/data-integrity/v1",
5      "https://schema.org/"
6    ],
7    "type": [ "VerifiableCredential", "TrustedIssuerCredential" ],
8    "issuer": "did:web:verifier.example.org",
9    "issuanceDate": "2024-01-01T00:00:00Z",
10   "expirationDate": "2026-01-01T00:00:00Z",
11   "credentialSubject": {
12     "id": "did:example:issuer2",
13     "type": "TrustedIssuer",
14     "verificationMethod": "did:key:zUC77BjGcGDVWf..." // Issuer2 の公開鍵
15   },
16   "proof": {
17     "type": "DataIntegrityProof",
18     "cryptosuite": "bbs-2023",
19     "verificationMethod": "did:web:verifier.example.org#key-1",
20     "proofPurpose": "assertionMethod",
21     "created": "2024-01-01T00:00:00Z",
22     "proofValue": "uqN6MS6QmTpe..."
23   }
24 }
```

VP のフォーマット 最後に，リスト 4 に Issuer-Hiding の方式で使用する VP の例を示す．ここでは，学生証 VC のうち，"type"と"award"のみを示し，かつ"issuer"の属性値と"proof"内の"verificationMethod"に書いてある Issuer の公開鍵を隠すために"_:b0"，"_:b1"という秘匿値を使用している．一方，Trusted Issuer VC のうち，Issuer の DID を示す"id"と公開鍵を示す"verificationMethod"を秘匿しながら示すために，学生証 VC で用いた"_:b0"，"_:b1"という秘匿値を使用している．これにより，学生証 VC の"issuer"の属性値である DID と Trusted Issuer VC の"id"の属性値である DID が同一であることを主張し，"verificationMethod"の属性値である公開鍵が同一であることを主張している．

リスト 4 Holder の作成した VP の例

```
1  {
2    // VP の例
3    "@context": [
4      "https://www.w3.org/2018/credentials/v1",
```

```
5     "https://www.w3.org/ns/data-integrity/v1",
6     "https://zkp-ld.org/context.jsonld",
7     "https://schema.org/",
8     "https://example.org/some-example-context"
9   ],
10  "type": "VerifiablePresentation",
11  "verifiableCredential": [
12    {
13      // 学生証VC(開示部分のみ)
14      "type": [ "VerifiableCredential", "StudentCredential" ],
15      "issuer": "_:b0", // Trusted Issuer VC の credentialSubject.id と等価な秘匿値
16      "issuanceDate": "2023-04-01T00:00:00Z",
17      "expirationDate": "2026-03-31T00:00:00Z",
18      "credentialSubject": {
19        "type": "Person",
20        "award": "Awesome Student"
21      },
22      "proof": {
23        "type": "DataIntegrityProof",
24        "cryptosuite": "bbs-2023",
25        "verificationMethod": "_:b1", // Trusted Issuer VC の credentialSubject.
              publicKey と等価な秘匿値
26        "proofPurpose": "assertionMethod",
27        "created": "2023-01-01T00:00:00Z"
28      }
29    },
30    {
31      // Trusted Issuer VC 1(開示部分のみ)
32      "type": [ "VerifiableCredential", "TrustedIssuerCredential" ],
33      "issuer": "did:web:verifier.example.org",
34      "issuanceDate": "2024-01-01T00:00:00Z",
35      "expirationDate": "2026-01-01T00:00:00Z",
36      "credentialSubject": {
37        "id": "_:b0",
38        "type": "TrustedIssuer",
39        "verificationMethod": "_:b1"
40      },
41      "proof": {
42        "type": "DataIntegrityProof",
43        "cryptosuite": "bbs-2023",
44        "verificationMethod": "did:web:verifier.example.org#key-1",
45        "proofPurpose": "assertionMethod",
46        "created": "2024-01-01T00:00:00Z"
47      }
48    }
49  ],
50  "proof": {
51    "type": "DataIntegrityProof",
52    "created": "2024-11-29T00:00:00Z",
53    "challenge": "some-challenge-string",
54    "cryptosuite": "bbs-2023",
55    "domain": "verifier.example.org",
56    "proofPurpose": "authentication",
57    "proofValue": "uomFhWQPQAgAA..."
58  }
59 }
```

7　まとめと今後の課題

ここまで，BBS 署名を用いた Issuer-Hiding AC の構成とその安全性について説明した．さらに，3 つの既存の Issuer-Hiding AC と提案手法を計算モデルや計算コスト，データサイズの面で比較・評価した．その結果，計算モデルでは提案手法のみが標準モデルを仮定しており，より現実的であることが示された．また，計算コストでは署名生成・検証やポリシー作成において，提案手法が既存手法よりも効率的であることが示された．さらに，データサイズにおいても，提案手法はいくつかの既存手法よりも小さいことが示された．一方で，同じく BBS 署名を用いた Katz らの手法と比較した場合，ポリシー検証，署名生成・検証においては提案手法が劣る結果となった．また，データサイズにおいても Katz らの手法が小さい結果となった．

提案手法は計算モデルといくつかの計算コストにおいて優れている．また，BBS 署名のみを用いているため，実装や運用が比較的容易である点も利点である．一方で，いくつかの計算コストとデータサイズにおいては Katz らの手法に劣る．ただし，提案手法も Katz らの手法も処理時間は最大でも 1, 2 秒程度であり，証明生成や証明検証においては数十ミリ秒程度であるため，実用上は大きな問題とはならないと考えられる．

今後の課題として，まず提案手法のさらなる効率化が挙げられる．特に，ポリシー作成において，発行元の数だけ乱数 e_i を生成しなければならないため，発行元の数が多い場合に効率が悪くなる．この点を改善することで，より効率的な Issuer-Hiding AC を実現できると考えられる．また，本稿で提案した Issuer-Hiding AC では Verifier が鍵生成を行なっているが，この鍵ペアを別のポリシー作成で使って良いか，あるいは何かしら問題があるかは別途検討が必要である．さらに，本稿では Issuer-Hiding AC を用いた VC と VP のデータフォーマットの構成例を示したが，実際のシステムに適用する際には，さらなる検討が必要である．

本稿が Issuer-Hiding AC の研究の一助となり，プライバシー保護に配慮したデジタル証明書システムの発展に寄与することを期待する．

謝辞

本稿の執筆にあたり，日頃より様々な形でご指導，助言をいただきました佐古和恵教授に深く感謝申し上げます．株式会社インターネットイニシアティブの山本暖氏には，様々な議論を通じて貴重なご助言を賜りました．心より感謝申し上げます．また，日頃から研究を支えていただいた佐古研究室の方々にも感謝申し上げます．

また，本研究の成果の一部は，公益財団法人セコム科学技術振興財団の特定領域研究「権限の濫用を抑止する技術的・社会的しくみに関する研究」によって得られたものです．ここに感謝の意を表します．

参考文献

[1] *Verifiable credentials data model v2.0 — w3.org*, [Accessed 2026-1-23]. [Online]. Available: https://www.w3.org/TR/vc-data-model-2.0/ (cit. on pp. 6, 7, 46).

[2] S. Tessaro and C. Zhu, "Revisiting bbs signatures," in *Advances in Cryptology – EUROCRYPT 2023*, C. Hazay and M. Stam, Eds., Cham: Springer Nature Switzerland, 2023, pp. 691–721, ISBN: 978-3-031-30589-4 (cit. on pp. 6, 11, 12, 31, 47).

[3] IRTF, *The bbs signature scheme*, [Accessed 2025-12-11]. [Online]. Available: https://datatracker.ietf.org/doc/draft-irtf-cfrg-bbs-signatures/ (cit. on p. 6).

[4] J. Groth, "Efficient fully structure-preserving signatures for large messages," in *Advances in Cryptology – ASIACRYPT 2015*, T. Iwata and J. H. Cheon, Eds., Berlin, Heidelberg: Springer Berlin Heidelberg, 2015, pp. 239–259, ISBN: 978-3-662-48797-6 (cit. on pp. 6, 7, 13, 23).

[5] J. Bobolz, F. Eidens, S. Krenn, S. Ramacher, and K. Samelin, "Issuer-hiding attribute-based credentials," in *Cryptology and Network Security: 20th International Conference, CANS 2021, Vienna, Austria, December 13-15, 2021, Proceedings 20*, Springer, 2021, pp. 158–178 (cit. on pp. 7, 17, 19, 22, 23, 29, 33–38).

[6] D. Pointcheval and O. Sanders, "Short randomizable signatures," in *Cryptographers' Track at the RSA Conference*, Springer, 2016, pp. 111–126 (cit. on pp. 7, 12, 47).

[7] O. Sanders and J. Traoré, "Compact issuer-hiding authentication, application to anonymous credential," *Proceedings on Privacy Enhancing Technologies*, 2024 (cit. on pp. 7, 25, 27, 33–38).

[8] J. Katz and M. Sefranek, *Issuer hiding for BBS-based anonymous credentials*, Cryptology ePrint Archive, Paper 2025/2080, 2025. [Online]. Available: https://eprint.iacr.org/2025/2080 (cit. on pp. 7, 27, 33–38, 45).

[9] V. Shoup, "Lower bounds for discrete logarithms and related problems," in *International Conference on the Theory and Applications of Cryptographic Techniques*, Springer, 1997, pp. 256–266 (cit. on pp. 7, 8).

[10] *Json-ld 1.1 — w3.org*, [Accessed 2025-12-29]. [Online]. Available: https://www.w3.org/TR/json-ld11/ (cit. on p. 7).

[11] S. D. Galbraith, K. G. Paterson, and N. P. Smart, "Pairings for cryptographers," *Discrete Applied Mathematics*, vol. 156, no. 16, pp. 3113–3121, 2008 (cit. on p. 8).

[12] G. Fuchsbauer, E. Kiltz, and J. Loss, "The algebraic group model and its applications," in *Annual International Cryptology Conference*, Springer, 2018, pp. 33–62 (cit. on p. 9).

[13] S. Goldwasser, S. Micali, and R. L. Rivest, "A digital signature scheme secure against adaptive chosen-message attacks," *SIAM Journal on computing*, vol. 17, no. 2, pp. 281–308, 1988 (cit. on p. 9).

[14] D. Boneh, X. Boyen, and H. Shacham, "Short group signatures," in *Advances in Cryptology – CRYPTO 2004*, M. Franklin, Ed., Berlin, Heidelberg: Springer Berlin Heidelberg, 2004, pp. 41–55, ISBN: 978-3-540-28628-8 (cit. on p. 11).

[15] M. H. Au, W. Susilo, and Y. Mu, "Constant-size dynamic k-taa," in *Security and Cryptography for Networks*, R. De Prisco and M. Yung, Eds., Berlin, Heidelberg: Springer Berlin Heidelberg, 2006, pp. 111–125, ISBN: 978-3-540-38081-8 (cit. on p. 11).

[16] S. Goldwasser, S. Micali, and C. Rackoff, "The knowledge complexity of interactive proof systems," *SIAM Journal on Computing*, vol. 18, no. 1, pp. 186–208, 1989 (cit. on p. 13).

[17] I. Damgård, "On Σ-protocols," *Lecture Notes, University of Aarhus, Department for Computer Science*, vol. 84, 2002 (cit. on p. 14).

[18] R. Cramer, I. Damgård, and B. Schoenmakers, "Proofs of partial knowledge and simplified design of witness hiding protocols," in *Annual International Cryptology Conference*, Springer, 1994, pp. 174–187 (cit. on p. 14).

[19] M. Blum, P. Feldman, and S. Micali, "Non-interactive zero-knowledge and its applications," in *Proceedings of the twentieth annual ACM symposium on Theory of computing*, 1988, pp. 103–112 (cit. on p. 15).

[20] A. Fiat and A. Shamir, "How to prove yourself: Practical solutions to identification and signature problems," in *Conference on the theory and application of cryptographic techniques*, Springer, 1986, pp. 186–194 (cit. on p. 15).

[21] M. Bellare and O. Goldreich, "On defining proofs of knowledge," in *Annual International Cryptology Conference*, Springer, 1992, pp. 390–420 (cit. on p. 15).

[22] D. Bernhard, O. Pereira, and B. Warinschi, "How not to prove yourself: Pitfalls of the fiat-shamir heuristic and applications to helios," in *International Conference on the Theory and Application of Cryptology and Information Security*, Springer, 2012, pp. 626–643 (cit. on pp. 16, 32).

[23] D. Chaum, "Security without identification: Transaction systems to make big brother obsolete," *Communications of the ACM*, vol. 28, no. 10, pp. 1030–1044, 1985 (cit. on p. 16).

[24] J. Camenisch and A. Lysyanskaya, "An efficient system for non-transferable anonymous credentials with optional anonymity revocation," in *International conference on the theory and applications of cryptographic techniques*, Springer, 2001, pp. 93–118 (cit. on p. 16).

[25] T. Ristenpart and S. Yilek, "The power of proofs-of-possession: Securing multiparty signatures against rogue-key attacks," in *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, Springer, 2007, pp. 228–245 (cit. on p. 18).

[26] M. Bellare and G. Neven, "Multi-signatures in the plain public-key model and a general forking lemma," in *Proceedings of the 13th ACM conference on Computer and communications security*, 2006, pp. 390–399 (cit. on p. 18).

[27] C. Hébant and D. Pointcheval, *Traceable constant-size multi-authority credentials*, Cryptology ePrint Archive, Paper 2020/657, 2020. DOI: 10.1007/978-3-031-14791-3_18. [Online]. Available: https://eprint.iacr.org/2020/657 (cit. on p. 23).

[28] J. Camenisch, M. Drijvers, and A. Lehmann, "Anonymous attestation using the strong diffie hellman assumption revisited," in *Trust and Trustworthy Computing: 9th International Conference, TRUST 2016, Vienna, Austria, August 29-30, 2016, Proceedings 9*, Springer, 2016, pp. 1–20 (cit. on pp. 24, 32).

[29] J. Camenisch and M. Stadler, "Efficient group signature schemes for large groups," in *Annual international cryptology conference*, Springer, 1997, pp. 410–424 (cit. on p. 29).

[30] S. Tessaro and C. Zhu, "Revisiting bbs signatures," *Cryptology ePrint Archive*, 2023 (cit. on pp. 31, 32).

[31] J. Camenisch and A. Lysyanskaya, "A signature scheme with efficient protocols," in *Security in Communication Networks*, S. Cimato, G. Persiano, and C. Galdi, Eds., Berlin, Heidelberg: Springer Berlin Heidelberg, 2003, pp. 268–289, ISBN: 978-3-540-36413-9 (cit. on p. 47).

[32] D. Fett, *SD-JWT-based Verifiable Credentials (SD-JWT VC) — ietf.org*, https://www.ietf.org/archive/id/draft-terbu-oauth-sd-jwt-vc-00.html, [Accessed 08-01-2024] (cit. on p. 47).